A Lightweight External Memory Engine to Bridge the Capability Gap Between On-Device and Cloud Language Models

[Internal Project]

Ibrahim Sudheysi Ali

166344

ICS 4C

Supervisor Name

Tiberius Tabulu

An Informatics and Computer Science Project Report Submitted to the School of Computing and Engineering Sciences in Partial Fulfillment of the Requirements for the Award of a Degree in Bachelor of Science in Informatics and Computer Science

School of Computing and Engineering Sciences

Strathmore University

Nairobi, Kenya

November 2026

Declaration and Approval

I declare that this work has not been previously submitted and approved for the award of a degree by this or any other University. To the best of my knowledge and belief, the report contains no material previously published or written by another person except where due reference is made in the report itself.

Student Name: Ibrahim Sudheysi Ali

Admission Number: 166344

Student Signature: _____ Date: _____

The project report of Ibrahim Sudheysi Ali has been reviewed and approved by Tiberius Tabulu, School of Computing and Engineering Sciences, Strathmore University.

Supervisor Signature: _____ Date: _____

Acknowledgment

I thank God for the strength and good health that carried me through this work. I am grateful to my supervisor, Mr. Tiberius Tabulu, whose patient guidance and honest feedback shaped this project from the proposal to the finished system. I thank the faculty of the School of Computing and Engineering Sciences at Strathmore University for the foundation they gave me, and the Sprint II review panel for asking for something a person can see and use. Finally, I thank my family and friends for their encouragement and support throughout this journey.

Abstract

Cloud language models are capable, but every question costs money, needs a connection, and sends personal data off the device. Models on a consumer laptop are private, offline, and free per question, but weaker on facts they have not memorised. Research indicates that this gap is largely missing knowledge, not missing reasoning ability.

This study designed, developed, and evaluated Burrow, a lightweight external memory engine that works beside a frozen on-device model. The memory is a vault of the person's own notes, documents, and code. On every question the engine decides whether the vault is needed, fits the amount of memory to the model within a token budget, and, on the command line, checks the sources an answer cites. Under the Design Science Research paradigm, the engine was built in Swift with an embedded database, a desktop application, a terminal chat, and a Model Context Protocol server. It was tested on 120 knowledge questions on three curated subjects and 50 reasoning questions from a public benchmark.

On the knowledge questions the on-device model, Gemma 4 12B, answered 9 alone and 102 with Burrow, while the cloud model, Claude, answered 40 alone and 109 with the same vault. On the reasoning questions it answered 47 alone, 7 with plain retrieval pasted in, and 48 with the engine, which left the memory out of all 50. Given the same vault, an on-device model can therefore approach a large cloud model on knowledge-bound tasks, provided the memory is withheld from reasoning questions.

Keywords: on-device language models, external memory, retrieval-augmented generation, capability gap, token budget

Table of Contents

Declaration and Approval ii
Acknowledgment iii
Abstract iv
Table of Contents v
List of Figures ix
List of Tables xi
List of Equations xii
List of Abbreviations xiii
Chapter 1: Introduction 1
1.1 Background Information 1
1.2 Problem Statement 2
1.3 Objectives 3
1.3.1 General Objective 3
1.3.2 Specific Objectives 3
1.4 Research Questions 4
1.5 Justification 4
1.6 Scope 5
1.7 Limitations and Delimitations 5
1.7.1 Limitations 5
1.7.2 Delimitations 5
Chapter 2: Literature Review 7
2.1 Introduction 7
2.2 On-Device Language Models and External Memory 7
2.2.1 The Rise of On-Device Models 7
2.2.2 Capability Gaps in Processing 8
2.2.3 The Scalability-Accuracy Trade-Off 8
2.2.4 Token-Efficient Retrieval 9
2.2.5 Evaluation and Risks of Persistent Memory 9
2.3 Memory Retrieval, and Compression Techniques 10
2.3.1 Techniques for Large Language Models (LLMs) 10
2.3.2 Techniques for Small Language Models (SLMs) 11
2.3.3 Techniques for On-Device Deployment 11
2.4 Related Works 11
2.4.1 MemGPT 12
2.4.2 Generative Agents 12

2.4.3 MemoryBank 13
2.4.4 HippoRAG 14
2.4.5 Mem0 14
2.4.6 Chroma 15
2.5 Gaps in Related Works 16
2.6 Conceptual Framework 17
Chapter 3: Methodology 19
3.1 Introduction 19
3.2 Research Paradigm 19
3.2.1 Data Acquisition 19
3.2.2 Data Preparation 20
3.2.3 Experimental Conditions 20
3.2.4 Measurement and Validation 20
3.3 Development Approach 21
3.4 System Analysis 22
3.5 System Design 22
3.5.1 Architectural Overview 23
3.5.2 Experimental Design 23
3.5.3 Datasets and Task Types 24
3.5.4 Evaluation Metrics 24
3.5.5 Ablation Procedure 24
3.5.6 Threats to Validity 24
3.6 Tools and Techniques 25
3.6.1 Swift 25
3.6.2 SQLite with FTS5 25
3.6.3 Ollama and Local Models 25
3.6.4 Hosted Inference API 25
3.6.5 Embedding Model 25
3.6.6 Chroma and LangChain 26
3.6.7 Model Context Protocol 26
3.6.8 Auxiliary Tools 26
3.7 Deliverables 26
3.7.1 The Burrow Memory Engine 26
3.7.2 The Desktop Application, Terminal Chat, and Protocol Server 26
3.7.3 The Evaluation Harness 26
3.7.4 Experimental Results 27
3.7.5 System Documentation 27

Chapter 4: System Analysis and Design 28
4.1 Introduction 28
4.2 System Requirements 28
4.2.1 Functional Requirements 28
4.2.2 Non-Functional Requirements 30
4.2.3 Data Requirements 30
4.2.4 Interface and Integration Requirements 31
4.3 System Analysis Diagrams 32
4.3.1 Use Case Diagram 32
4.3.2 Sequence Diagrams 34
4.3.3 Activity Diagram 36
4.3.4 Class Diagram 37
4.4 System Design Diagrams 38
4.4.1 System Architecture 38
4.4.2 Engine Steps and Retrieval Design 39
4.4.3 Logical Database Schema 41
4.4.4 Wireframes 41
4.4.5 Deployment Diagram 44
Chapter 5: System Implementation, Testing and Evaluation 46
5.1 Introduction 46
5.2 Implementation Environment 46
5.2.1 Hardware and Deployment Environment 46
5.2.2 Software, Services and Configuration 47
5.2.3 Deployment and Reproducibility 48
5.3 Dataset Description 49
5.3.1 Description of the Dataset 49
5.3.2 Data Preparation and Partitioning 49
5.4 System Implementation 50
5.4.1 The Vault and Its Index 50
5.4.2 Decide 51
5.4.3 Fit 52
5.4.4 Answer and Check 53
5.4.5 Skills and Learn 55
5.4.6 Serving Other Models and Tools 55
5.4.7 Deviations from the Approved Design 57
5.5 Model Selection and Evaluation 57
5.5.1 Baselines and Set-ups 57

5.5.2 Evaluation Metrics 58
5.5.3 Results on the Main Model 59
5.5.4 Results on the Small Model 61
5.5.5 Ablation, Budget and Speed 62
5.5.6 Error Analysis 64
5.5.7 A Coding Trial Against the Cloud Model 65
5.6 System Testing 66
5.6.1 Test Strategy and Acceptance Criteria 66
5.6.2 Test Cases, Defects and Retesting 67
5.7 Version Control and Implementation Evidence 70
5.8 Author Oversight and Use of AI-Assisted Tools 70
5.9 Evaluation Results, Discussion and Limitations 71
5.9.1 Objective i: The Capability Gap and Existing Techniques 71
5.9.2 Objective ii: The Design of the Engine 71
5.9.3 Objective iii: The Development of the Engine 71
5.9.4 Objective iv: The Fraction of the Gap Closed and Its Cost 72
5.9.5 Objective v: The Token Budget 73
5.9.6 Limitations 74
References 76
Appendices 77
Appendix 1: Gantt Chart 77

List of Figures

Figure 2.1: The MemGPT virtual context architecture. Adapted from Packer et al. (2023) ... 12
Figure 2.2: The Smallville environment of Generative Agents. Source: Park et al. (2023) 13
Figure 2.3: The MemoryBank framework with updating and forgetting. Source: Zhong et al. (2024) ... 13
Figure 2.4: HippoRAG 2 offline indexing and online retrieval. Source: Gutiérrez et al. (2025) ... 14
Figure 2.5: The Mem0 memory layer within an agent stack. Source: Mem0 documentation, https://docs.mem0.ai ... 15
Figure 2.6: The architecture of distributed Chroma. Source: Chroma (2026), https://docs.trychroma.com/reference/architecture/distributed ... 15
Figure 2.7: Conceptual framework of the study, drawn by the author ... 18
Figure 3.1: Incremental development approach for the Burrow engine, drawn by the author 22
Figure 4.1: Use case diagram of Burrow ... 33
Figure 4.2: Sequence diagram of a question asked through the engine ... 35
Figure 4.3: Sequence diagram of an outside model preparing a task through MCP ... 36
Figure 4.4: Activity diagram of the engine's loop ... 37
Figure 4.5: Class diagram of the engine ... 38
Figure 4.6: Architecture of Burrow ... 39
Figure 4.7: The five steps of the engine and the mechanisms of the proposal ... 40
Figure 4.8: Logical schema of the index ... 41
Figure 4.9: Wireframe of the main window with the Ask panel ... 42
Figure 4.10: Wireframe of the note editor with its links ... 43
Figure 4.11: Wireframe of the step where the person keeps what was decided ... 43
Figure 4.12: Wireframe of the terminal chat ... 44
Figure 4.13: Deployment diagram of Burrow ... 45
Figure 5.1: The graph of a vault in the desktop app, coloured by folder ... 51
Figure 5.2: A note in the desktop app, with its links and properties ... 51
Figure 5.3: The engine answering a question about code and reporting its decision, the skill used, the files read and the citation check ... 52
Figure 5.4: A sum in the desktop app, answered with the vault left out ... 52
Figure 5.5: Measuring a model once with burrow fit, and the skills in the vault ... 53
Figure 5.6: A question answered from the vault in the desktop app, with the engine's decision and the files read ... 53
Figure 5.7: The files read for an answer lighting up in the graph ... 54
Figure 5.8: A question the excerpts could not answer, and the model saying so ... 54
Figure 5.9: A skill as it is stored in the vault ... 55
Figure 5.10: An outside client calling the tool vault_prepare through the Model Context Protocol ... 56
Figure 5.11: Burrow as the retriever in a LangChain pipeline ... 56
Figure 5.12: Answers right by set-up and task type, Gemma 4 12B and the cloud model ... 59
Figure 5.13: Answers right by set-up and task type, Qwen3.5 4B and the cloud model ... 62
Figure 5.14: Knowledge answers right against tokens read, as the amount of the vault shown to Gemma 4 12B grows ... 63
Figure 5.15: Median time of one lookup on the Django index, before and after the search was rebuilt ... 64
Figure 5.16: The same three briefs built by Gemma 4 12B alone, by Gemma with a skill and a note of known failures, and by Claude alone ... 66

Figure 5.17: The history of the engine's source, the tags and the folders of the repository 70
Figure A1.1: Project work plan, May to November 2026 77

List of Tables

Table 2.1: Comparison of representative memory systems 16
Table 3.1: Two records of the evaluation data 20
Table 4.1: Functional requirements 29
Table 4.2: Non-functional requirements 30
Table 4.3: Data requirements 31
Table 4.4: Interface and integration requirements 32
Table 4.5: Use case description: Ask a question 34
Table 5.1: Hardware and deployment environment 47
Table 5.2: Software, models and services 48
Table 5.3: The evaluation vault and its questions 49
Table 5.4: The six set-ups compared 58
Table 5.5: Answers right by set-up and task type, Gemma 4 12B 59
Table 5.6: Fraction of the capability gap closed (Equation 3.1), against the cloud model alone 60
Table 5.7: Cost of an answer on knowledge questions: tokens read, seconds, and whether the evidence was retrieved 61
Table 5.8: How much of the vault the model is shown, against what it gains, Gemma 4 12B, knowledge questions 63
Table 5.9: Examples of answers from Gemma 4 12B with the engine 65
Table 5.10: Test cases and results 67
Table 5.11: How each specific objective was met 74

List of Equations

Equation 3.1: Fraction of the capability gap closed 23
Equation 4.1: Cosine similarity between a question and a piece 40
Equation 4.2: Merged score of a piece 40
Equation 4.3: Score of a neighbouring file during a walk 41
Equation 5.1: Accuracy of a set-up 58
Equation 5.2: Exact two-sided sign test on paired answers 58

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
BM25	Best Matching 25
DSR	Design Science Research
FTS5	Full-Text Search version 5 (SQLite)
GSM8K	Grade School Math 8K
JSON	JavaScript Object Notation
JSON-RPC	JSON Remote Procedure Call
LLM	Large Language Model
MCP	Model Context Protocol
OOAD	Object-Oriented Analysis and Design
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
SLM	Small Language Model
SQL	Structured Query Language
SSD	Solid-State Drive
UML	Unified Modeling Language

Chapter 1: Introduction

1.1 Background Information

Language models have become the foundation of modern conversational software, powering assistants that answer questions, summarise documents, and help with everyday tasks. The most capable of these models are served from the cloud on specialised hardware. Their capability comes at a cost: every query travels over a network, personal data leaves the user's device, and each request carries a price. Z. Zhang et al. (2024) analysed real ChatGPT conversations and interviewed 19 users, and found that people constantly trade privacy against usefulness. Empirical work on scaling has shown that the loss of a language model falls predictably with its size, training data, and compute (Kaplan et al., 2020), which is precisely why the strongest models are also the largest and the least practical to run locally.

A parallel movement has formed around small language models that run directly on consumer hardware such as laptops and phones. Open-weight families such as Qwen2.5 (Yang et al., 2024), Llama 3 (Grattafiori et al., 2024), Phi-3 (Abdin et al., 2024), and the 2026 releases Gemma 4 (Gemma Team, 2026) and Qwen3.5 (Qwen Team, 2026) include models that fit in the memory of an ordinary computer, and Apple now ships a foundation model on its own devices (Gunter et al., 2024). J. Xu et al. (2024) named lower latency, data that stays on the device, and personalisation among the reasons for running a model locally. Lu et al. (2024) measured 70 open models of up to five billion parameters and reported both what they can do and what they cost to run on a device.

The most consistent weakness of these models is factual knowledge. Mallen et al. (2023) showed that language models struggle with less popular facts, that retrieval of external text helps significantly in exactly those cases, and that making models bigger barely improves the long tail. Kandpal et al. (2023) explained why: a model's accuracy on a factual question depends on how many training documents mention the fact. A person's own notes, documents, and code are the extreme case, because a public model is unlikely to have seen them in training. This finding suggests a practical opportunity. If much of the gap between small and large models is missing knowledge rather than missing reasoning ability, then supplying the right knowledge at the right time, from outside the model, might recover a meaningful share of the gap without touching the model at all.

1.2 Problem Statement

A person who wants a private, offline, low-cost assistant should be able to run a language model on their own machine and rely on it as they would rely on a cloud service. In practice that person must accept a small on-device model, and with it a marked drop in capability, most visibly on questions that require facts the model has not memorised. The two standard remedies do not fit the on-device setting. A larger model no longer fits in the memory of the device, and fine-tuning the small model needs extensive resources (Soudani et al., 2024) and must be repeated whenever the knowledge changes.

Retrieval offers a third path: leave the model frozen and supply relevant text at question time. The literature has established the ingredients separately. Retrieval can substitute for model scale on knowledge-heavy tasks (Borgeaud et al., 2022; Izacard et al., 2023; Lewis et al., 2020), memory systems can accumulate information across long interactions (Gutiérrez et al., 2024; Packer et al., 2023; Zhong et al., 2024), and prompts can be compressed so that less text carries the same meaning (Jiang et al., 2023). Retrieved text can also lower accuracy when it is irrelevant (Shi et al., 2023; Yoran et al., 2024), so a memory beside a small model must decide when to supply text and how much. The nearest published measure, in which a small on-device model recovered 87% of a cloud model's performance, depended on working with the cloud model throughout (Narayan et al., 2025). No existing work reports what fraction of the gap a memory engine beside a frozen small model closes on a real consumer device, on which kinds of tasks, and at what cost in tokens and latency.

Without that answer, designers of on-device assistants are forced to choose between privacy and capability without evidence, and the field cannot say whether memory engineering is a genuine substitute for model scale or only a partial remedy.

The specific solution this study developed is Burrow, a lightweight external memory engine that sits beside a frozen on-device language model on a consumer laptop. Burrow addresses the problem by giving the model an external, persistent memory rather than larger weights. It keeps a vault of the person's own notes, documents, and code, decides for each question whether that memory is needed, retrieves the most relevant passages, fits them to the amount the model can read well, and proposes compact notes for the person to keep, while the model's parameters are never changed. The study then measured how much of the capability gap this memory closed, separately for knowledge-bound and reasoning-bound tasks, and at what cost in tokens and latency.

1.3 Objectives

This section states the general objective of the study and breaks it down into the specific objectives that guided the work.

1.3.1 General Objective

The general objective of this study was to design, develop, and validate a lightweight memory engine for a frozen small language model on a consumer laptop, in order to quantify how much of the capability gap between small on-device and large cloud language models such a memory engine can close, by task type, under explicit token and latency accounting.

1.3.2 Specific Objectives

i. To investigate the capability gaps between large, small, and on-device language models, together with existing memory, retrieval, and compression techniques and the interaction between a model's internal (parametric) and external (retrieved) knowledge, so as to establish their applicability and limitations in the on-device setting.

ii. To design a lightweight memory engine for a frozen small language model, comprising a write policy that distils interactions into compact stored notes, a retrieval policy that ranks notes by similarity, recency, and importance, and a token budgeting mechanism that controls how much memory the model reads.

iii. To develop the designed memory engine on a consumer laptop with no dedicated graphics hardware, using an embedded database as the memory store.

iv. To evaluate the implemented engine experimentally by measuring the fraction of the capability gap between a small local model and a large cloud model that the engine closes, separately for knowledge-bound and reasoning-bound tasks, together with the token cost and latency of every answer.

v. To determine the token budget that maximises the fraction of the capability gap closed per token, so that the engine can be configured for the best accuracy-to-cost trade-off on a resource-constrained device.

1.4 Research Questions

i. What are the capability gaps between large, small, and on-device language models, and how applicable and limited are existing memory, retrieval, and compression techniques in the on-device setting, given the interaction between a model's internal and retrieved knowledge?

ii. How can a lightweight memory engine for a frozen small language model be designed, in terms of its write policy, its retrieval policy, and its token budgeting mechanism?

iii. How can the designed memory engine be developed on a consumer laptop with no dedicated graphics hardware, using an embedded database as the memory store?

iv. What fraction of the capability gap between a small local model and a large cloud model does the engine close on knowledge-bound and on reasoning-bound tasks, and at what cost in tokens and latency?

v. What token budget maximises the fraction of the capability gap closed per token, and how does gap closure change as the budget increases?

1.5 Justification

On-device models are now capable enough to be useful, yet they remain weakest on the knowledge that people most want to keep private, and no study has measured how far an external memory narrows that gap on a consumer device. The study matters for three audiences. For users who handle private material, a more capable on-device assistant keeps that material on their own machine. Mireshghallah et al. (2024) found personally identifiable information in 16% of code-editing conversations with commercial chat models, and in Kenya personal data is protected by statute (Data Protection Act, 2019), with data and AI sovereignty reflected in the national AI strategy (Ogonjo, 2025). For developers of on-device assistants, the study provides a measured account of how far memory engineering can stand in for model scale, allowing informed decisions about where to invest effort. For the research community, the study contributes an evaluation method, the gap-closed measure reported per task type with token and latency accounting on consumer hardware, against which later memory systems can be compared. This work helps people who need a private assistant by giving a small local model access to what they know, while advancing research on memory for on-device language models.

1.6 Scope

The deployment target was a consumer computer with no dedicated graphics card, of the kind a person already owns. One such laptop was the platform on which the engine ran and on which all latency was measured, and Section 5.2.1 gives its specification. The on-device models were open-weight models run locally through Ollama, with Gemma 4 12B as the main model and Qwen3.5 4B as the small model. The large reference model, Claude (Anthropic, 2026c), was reached through a cloud API, never ran locally, and served only as the upper anchor against which the gap was measured. Evaluation used 170 English questions: 120 knowledge questions on three curated subjects, with single-fact and multi-fact forms, and 50 mathematical reasoning questions from a public benchmark, so that knowledge-bound and reasoning-bound performance could be separated. The system delivered was the memory engine, its evaluation harness, and four ways to use the engine: a command-line tool, a native desktop application, a terminal chat, and a Model Context Protocol server. The work ran from the approved proposal of June 2026 to the final defence of November 2026, and it did not cover mobile phones, languages other than English, images or audio, or use by several people at once.

1.7 Limitations and Delimitations

1.7.1 Limitations

The study operated under constraints that shape how the findings should be interpreted. First, latency measured on one consumer laptop may differ on other machines, so the device results are indicative rather than universal. Second, the large reference model is a hosted service whose behaviour may change over time, which can affect the ceiling used to calculate the capability gap. Third, the questions on the curated subjects were written from the source documents by Claude Code agents and not by people, so the results may not transfer equally to every live use. Fourth, the study was limited by the cost and time required to test memory on many models, devices, languages, and modalities within one undergraduate project.

1.7.2 Delimitations

The delimitations respond directly to those constraints. To address the hardware limitation, the study deliberately fixed one consumer laptop as the target device and reported the full hardware specification and the seconds taken by every answer. To address ceiling instability, the study pinned and logged one version of the cloud model for the whole experiment. To

address the generality of the questions, the study used three unrelated subjects and one public benchmark, and reported single-fact, multi-fact, and mathematical reasoning results separately rather than one broad score. To address the limit of cost and time, the study tested two on-device models on one device in one language, did not train or fine-tune any language model, did not port the engine to mobile phones, and did not evaluate multilingual or multimodal memory. For the same reason, the graph-based retrieval layer that the proposal held back as optional was built in a light form, as a walk over the links that already exist between files.

Chapter 2: Literature Review

2.1 Introduction

This chapter reviews the research that the study builds on, in order to ground the problem of the capability gap between small on-device and large cloud language models in existing work. Section 2.2 surveys the current state of the field: the rise of small models, what the capability gap consists of, retrieval as a substitute for scale, the efficient use of what a model reads, and the evaluation and security of persistent memory. Section 2.3 reviews memory, retrieval, and compression techniques at each scale of model. Section 2.4 examines specific related systems, Section 2.5 identifies the gaps those systems leave, and Section 2.6 closes with the conceptual framework that guided the study. The sources are mainly peer-reviewed conference and journal papers, with technical reports for the models and official documentation for the software products. Most were published between 2023 and 2026, and older sources were kept where they introduced a method or theory that this study uses.

2.2 On-Device Language Models and External Memory

This section reviews the current state of the field and addresses the first specific objective of the study.

2.2.1 The Rise of On-Device Models

Scaling studies established that the loss of a language model falls smoothly with its parameter count, training data, and compute (Kaplan et al., 2020), which drove the field toward ever larger models served from the cloud. A counter-movement followed. Qwen2.5 (Yang et al., 2024), Llama 3, whose smallest model has 8 billion parameters (Grattafiori et al., 2024), and Phi-3 (Abdin et al., 2024) were published as open-weight models small enough for an ordinary laptop, and Gunter et al. (2024) described the model that Apple ships on its own devices. J. Xu et al. (2024) reviewed this movement and named lower latency, data that stays local, and personalisation among its reasons.

Later generations reach similar quality at smaller sizes. Liu et al. (2024) showed that careful architecture lets models under one billion parameters come close to a seven-billion-parameter model on API-calling tasks, and the Gemma 3 report stated that its 4B model was competitive with the previous generation's 27B model (Gemma Team, 2025). The families released in 2026, Gemma 4 (Gemma Team, 2026) and Qwen3.5 (Qwen Team, 2026), include the two

models hosted in this study. Lu et al. (2024) measured 70 open small models and reported their ability together with their latency and memory use on a device.

2.2.2 Capability Gaps in Processing

The most important evidence for this study concerns the composition of the gap. Mallen et al. (2023) tested ten models on 14,000 factual questions and found that models struggle with less popular facts and that scaling barely helps in the long tail. Sun et al. (2024) saw the same pattern in 16 public models, which were far from perfect on the rarer entities. Kandpal et al. (2023) traced the cause to training data: accuracy on a fact depends on how many training documents mention it, and closing the long tail by size alone would need many orders of magnitude more parameters. A large part of what users buy with model scale is therefore memorised knowledge, and memorised knowledge is the one ingredient that can also be supplied from outside the model. This study treated knowledge-bound and reasoning-bound tasks separately for that reason.

2.2.3 The Scalability-Accuracy Trade-Off

Letting a model read relevant external text at answer time can substitute for parameters. Retrieval-augmented generation paired a generator with an index of documents that it reads at answer time (Lewis et al., 2020), building on dense passage retrieval, which finds relevant text by comparing embedding vectors rather than keywords (Karpukhin et al., 2020). Borgeaud et al. (2022) made the substitution explicit: their RETRO model, retrieving from a database of two trillion tokens, reached performance comparable to GPT-3 and Jurassic-1 with 25 times fewer parameters. Izacard et al. (2023) showed with Atlas that a retrieval-augmented model could beat a 540-billion-parameter model on NaturalQuestions while using 50 times fewer parameters. In the study by Mallen et al. (2023), retrieval-augmented models largely outperformed models that were orders of magnitude larger, and Soudani et al. (2024) found across twelve models that retrieval beat fine-tuning by a large margin on the least popular facts.

The trade-off has a second side, because retrieved text can harm an answer. Yoran et al. (2024) found that retrieval sometimes lowered accuracy across five question-answering benchmarks, and Cuconasu et al. (2024) identified the harmful passages as those a retriever scores highly although they do not contain the answer. Reasoning tasks are especially exposed. Shi et al. (2023) saw performance fall dramatically when irrelevant information was added to grade-school mathematics problems, and Mirzadeh et al. (2025) reported drops of

up to 65% from a single clause that only seemed relevant. More context is not simply better either. Levy et al. (2024) found a notable drop in reasoning at input lengths far shorter than a model's maximum, Hsieh et al. (2024) found that only half of 17 long-context models kept satisfactory performance at 32,000 tokens, and Jin et al. (2025) showed that output quality first improves and then declines as the number of retrieved passages rises. None of these results addresses the on-device setting, where the index and the model must fit on a consumer machine and every token read costs measurable time.

2.2.4 Token-Efficient Retrieval

Every token a model reads costs time on a device, so what is put in front of the model matters as much as its relevance. Research on prompt compression has shown that the text supplied to a model carries substantial redundancy. Jiang et al. (2023) demonstrated with LLMLingua that prompts can be compressed up to about twenty times with little loss in performance. A complementary line compresses context into learned representations: Chevalier et al. (2023) taught models to summarise long contexts into compact vectors, and Ge et al. (2024) trained an in-context autoencoder that reached about four times context compression on Llama. These methods change the text or the model after the passages have been chosen. The lesson this study took is that the first saving on a device comes from selecting less: reading the memory only when it is needed, and showing the model no more than it reads well. Compression has so far been applied to prompts and contexts at inference time, and not to the question of how a token budget interacts with capability gap closing on a device.

2.2.5 Evaluation and Risks of Persistent Memory

Benchmarks for conversational memory inform the evaluation design of this study. Maharana et al. (2024) introduced LoCoMo, a benchmark of very long multi-session conversations, and found that models lag far behind human recall of conversation history. Wu et al. (2025) introduced LongMemEval, 500 questions embedded in long chat histories, and found that commercial assistants lose about 30% accuracy when information must be remembered across sustained interaction. They also divided memory design into indexing, retrieval, and reading stages. Trust in what a model does with its context is a further concern: Joren et al. (2025) found that smaller models often hallucinate or abstain even when the context is sufficient, and Gao et al. (2023) found that even the best models lacked complete citation support half of the time on one benchmark. These studies show that memory is an unsolved

problem even for large cloud systems, but none measures memory as an instrument for closing the gap between a small and a large model.

A memory store that accepts writes is also an attack surface. Greshake et al. (2023) showed that instructions hidden in retrieved data can take over an application. Chen et al. (2024) showed with AgentPoison that a small number of crafted records in an agent's memory can steer it when a trigger appears, with benign instructions left untouched, and Dong et al. (2025) showed that an attacker who can only send queries can plant malicious records. Hou et al. (2026) catalogued 16 threat scenarios for servers of the Model Context Protocol. These findings motivated two design choices in Burrow's chat: each proposed note must be backed by the person's own words, and nothing is written to the vault until the person agrees. A full security evaluation was beyond the scope of this study.

2.3 Memory Retrieval, and Compression Techniques

The preceding section established the capability gap and the principle that external knowledge can substitute for parameters. This section surveys the techniques that make that substitution work in practice. The review is organised by the scale of the host model, because the constraints, and therefore the appropriate techniques, change as the model shrinks and moves onto the device.

2.3.1 Techniques for Large Language Models (LLMs)

At the scale of large cloud models, retrieval and compression are optimisation choices rather than necessities, and the literature has focused on maximising answer quality. Fan et al. (2024) surveyed the retrieval architectures that followed the original work of Lewis et al. (2020). Memory extends retrieval from documents to what an agent keeps over time. Sumers et al. (2024) described a language agent as having working, episodic, semantic, and procedural memory, and Hu, Liu, et al. (2025) separated agent memory from retrieval-augmented generation and classified it by form, function, and dynamics. Luo et al. (2026) described three stages of agent memory, storage, reflection, and experience, and Huang et al. (2026) treated portable agent skills as part of memory. Cloud assistants already apply these ideas: Claude Code reads a codebase and acts on it (Anthropic, 2026a) and keeps a project memory in plain files (Anthropic, 2026b). These techniques assume cloud-scale compute and do not account for the memory and latency budgets of a device.

2.3.2 Techniques for Small Language Models (SLMs)

As models shrink, retrieval shifts from optional to essential, because the model can no longer rely on what it has memorised. Wang et al. (2025) surveyed how small language models are defined, acquired, enhanced, and applied, and how they work with larger models. Patra et al. (2026) surveyed on-device small models across architecture, compression, applications, and open challenges. A central question at this scale is when to retrieve at all. Self-RAG needs a specially trained model that judges when retrieval is useful (Asai et al., 2024), and Adaptive-RAG needs a trained classifier that routes a question to no retrieval, one-step retrieval, or multi-step retrieval (Jeong et al., 2024). Moskvoretskii et al. (2025) compared 35 such methods and found that simple uncertainty estimates often matched complex pipelines in answer quality. For a frozen model on a device this finding matters, because a method that needs no training can be applied to any model that a person chooses to run.

2.3.3 Techniques for On-Device Deployment

On the device itself, retrieval must respect a fixed hardware budget, so inexpensive rankings are preferred. Ranking by word match uses the BM25 function (Robertson & Zaragoza, 2009), and reciprocal rank fusion merges a word ranking and a meaning ranking without training (Cormack et al., 2009). Bruch et al. (2023) showed that a tuned weighted sum of scores can outperform this fusion, but it needs a training set. The form of the memory also matters. Obsidian keeps a vault as ordinary Markdown files on the person's own disk (Obsidian, 2025a) and draws the links between notes as a graph (Obsidian, 2025b), so the person owns the memory and can read it without any model. Narayan et al. (2025) came nearest to the question of this study: an on-device model reading local documents recovered 87% of a cloud model's performance, and 97.9% when the cloud model split the task into small local jobs, but it depended on the cloud throughout. Taken together, this work shows that the techniques exist and are individually effective. It does not provide a measured account of how much of the capability gap a memory engine closes on its own, by task type, on consumer hardware.

2.4 Related Works

This section examines specific systems that implement memory for language models, from operating-system inspired context management to brain-inspired associative retrieval, production memory layers, and vector databases. Table 2.1 summarises the comparison.

2.4.1 MemGPT

Packer et al. (2023) framed the memory problem in operating-system terms. The model's limited context window is treated as fast main memory and an external store as disk, and the system pages information in and out as the conversation demands, allowing a fixed-context model to sustain interactions far longer than its window. The framing is influential because it makes explicit that what the model reads at each step is a policy decision, which is the decision this study's token budget controls. MemGPT manages context length, however. It does not target capability gain for a small model, and it does not account for token cost or device latency. Figure 2.1 illustrates the two-tier architecture.

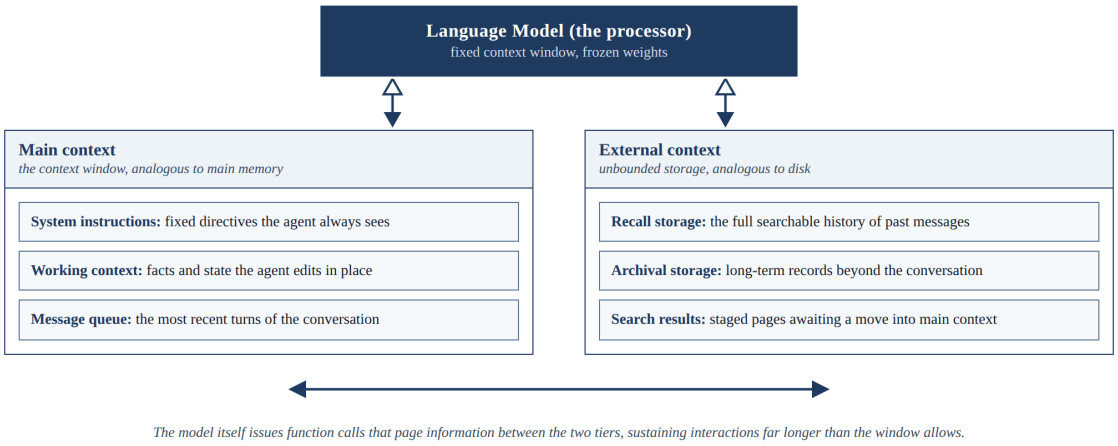


Figure 2.1: The MemGPT virtual context architecture. Adapted from Packer et al. (2023)

2.4.2 Generative Agents

Park et al. (2023) gave simulated agents a memory stream of timestamped observations and retrieved from it by combining three scores: relevance to the current situation, recency, and importance. The agents produced believable long-running behaviour in a simulated town, and the three-part score shaped the retrieval policy first proposed for this study. Zhao et al. (2024) later showed that insights written in natural language and recalled at inference improve an agent without any parameter updates. Generative Agents was designed for simulation rather than question answering, and its memory records are verbose natural-language entries, so it does not answer the efficiency questions a device imposes. Figure 2.2 shows the simulated town in which the agents live, work, and remember.



Figure 2.2: The Smallville environment of Generative Agents. Source: Park et al. (2023)

2.4.3 MemoryBank

Zhong et al. (2024) added a model of forgetting. MemoryBank strengthens or weakens stored memories over time in the spirit of the Ebbinghaus forgetting curve, so that what the system retains reflects both the time elapsed and the relative significance of a memory. The contribution is a principled answer to what should be kept, and Hu, Wang, and McAuley (2025) later named selective forgetting as one of four competencies of a memory agent. MemoryBank was evaluated through a companion chatbot, SiliconFriend, on real and simulated dialogs, and not on capability gain, token cost, or latency. Figure 2.3 shows the memory bank, the forgetting curve, and retrieval into conversation.

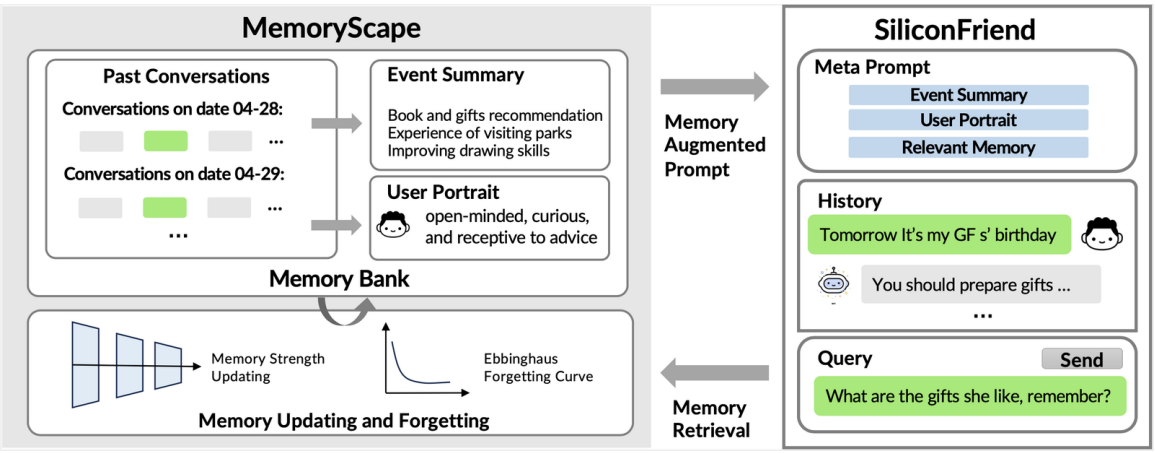


Figure 2.3: The MemoryBank framework with updating and forgetting. Source: Zhong et al. (2024)

2.4.4 HippoRAG

Plain similarity search retrieves passages that resemble the question, but some questions require connecting facts that do not resemble the question at all. Cognitive science offers a model for this in spreading activation, where recalling one concept activates linked concepts (Collins & Loftus, 1975), and in the hippocampal indexing theory, where a compact index links the distributed traces of an experience (Teyler & DiScenna, 1986). Gutiérrez et al. (2024) built these ideas into HippoRAG: a language model extracts entities from stored passages into a knowledge graph, and a Personalized PageRank walk spreads relevance from the question to connected passages. HippoRAG improved multi-hop question answering by up to 20%, and its single-step retrieval was 10 to 30 times cheaper than iterative retrieval such as IRCoT. A language model must build the graph, however, as in GraphRAG (Edge et al., 2024) and A-Mem (W. Xu et al., 2025), and Gutiérrez et al. (2025) noted that graph methods fall considerably below standard retrieval on basic factual memory. This study therefore searched the text first and followed links that already exist between files. Figure 2.4 shows the pipeline of its successor, HippoRAG 2.

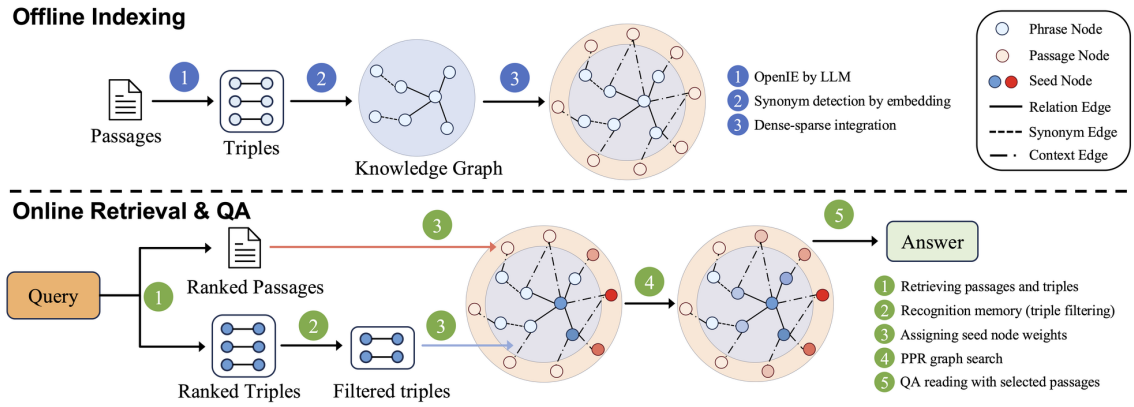


Figure 2.4: HippoRAG 2 offline indexing and online retrieval. Source: Gutiérrez et al. (2025)

2.4.5 Mem0

Production systems have begun packaging these ideas. Chhikara et al. (2025) presented Mem0, which extracts the salient facts from a conversation, consolidates them, and retrieves them at question time. On the LoCoMo benchmark, the authors reported a 26% relative improvement over OpenAI's memory on a metric judged by a language model. Its existence is evidence that the engineering direction is commercially credible. Mem0 writes memories automatically, however, and it was evaluated on conversational recall, so it does not measure gap closing per task type. Figure 2.5 shows where the memory layer sits in an agent stack.

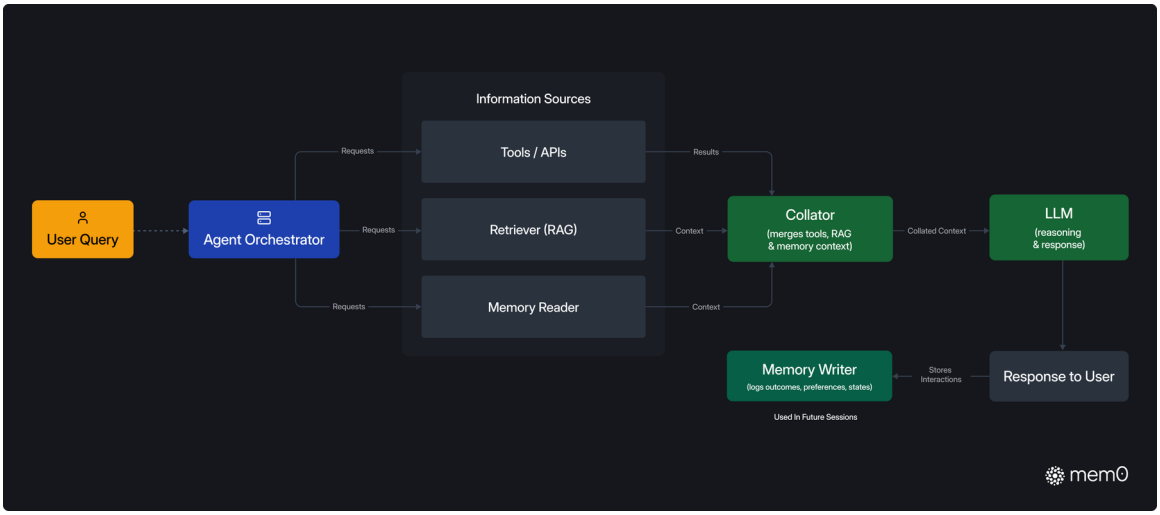


Figure 2.5: The Mem0 memory layer within an agent stack. Source: Mem0 documentation, https://docs.mem0.ai

2.4.6 Chroma

Chroma is an open-source vector database (Chroma, 2026). Documents are cut into pieces, each piece is stored with an embedding vector and its text, and at question time the pieces nearest to the question are pasted into the prompt. It was the baseline of this study because it is what a developer would build without a memory engine. A vector database is a store and not an engine: it answers every question with its nearest pieces, and highly ranked pieces without the answer are the ones that hurt (Cuconasu et al., 2024). Chroma's own researchers reported that reliability falls as input grows (Hong et al., 2025). Figure 2.6 shows the distributed architecture.

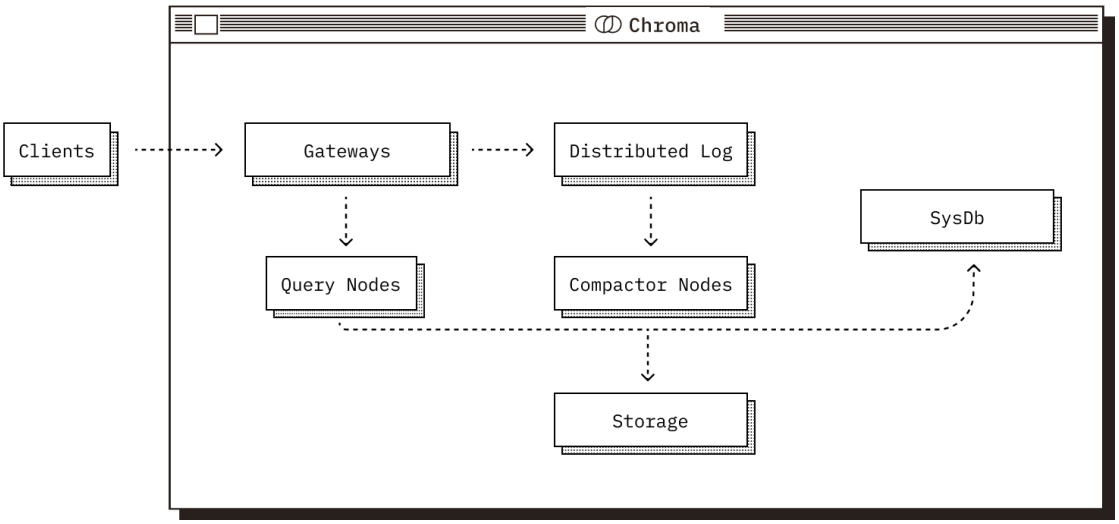


Figure 2.6: The architecture of distributed Chroma. Source: Chroma (2026), https://docs.trychroma.com/reference/architecture/distributed

Table 2.1: Comparison of representative memory systems

System	Core idea	Strength	Limitation for the on-device setting
MemGPT (Packer et al., 2023)	Pages context in and out of the window like an operating system	Sustains very long interactions on a fixed window	Manages context length; does not target capability gain or token cost
Generative Agents (Park et al., 2023)	Memory stream scored by relevance, recency, and importance	Simple, effective retrieval scoring	Designed for simulation; verbose natural-language records
MemoryBank (Zhong et al., 2024)	Memories strengthen and decay over time	Principled forgetting	No accounting of token cost or device latency
HippoRAG (Gutiérrez et al., 2024)	Knowledge-graph index walked with Personalized PageRank	Strong multi-hop recall in a single retrieval step	A language model must build the graph, which is heavy for a device
Mem0 (Chhikara et al., 2025)	Salient facts extracted and consolidated into a managed store	Production-ready; gains reported on a long-conversation benchmark	Writes memories automatically; gap closing per task type not measured
Chroma (Chroma, 2026)	Vector database queried for the pieces nearest to the question	Simple, runs locally, the standard pipeline	A store, not an engine; same retrieval for every question and every model

2.5 Gaps in Related Works

The reviewed literature establishes each ingredient separately. Retrieval can substitute for model scale on knowledge-heavy tasks (Borgeaud et al., 2022; Izacard et al., 2023; Lewis et al., 2020), and it helps most on the long-tail knowledge where small models fail (Kandpal et al., 2023; Mallen et al., 2023). Memory systems can accumulate, organise, score, and forget information across long interactions (Chhikara et al., 2025; Gutiérrez et al., 2024; Packer et al., 2023; Park et al., 2023; Zhong et al., 2024). Text supplied to a model can be heavily compressed with little loss (Chevalier et al., 2023; Ge et al., 2024; Jiang et al., 2023). Methods that decide when to retrieve exist, but they need a specially trained model or a trained classifier (Asai et al., 2024; Jeong et al., 2024), and benchmarks expose how poorly current systems remember long interactions (Maharana et al., 2024; Wu et al., 2025).

What is missing is the combination and the measurement. As Table 2.1 shows, the systems in Section 2.4 are evaluated on recall or task success, and none on gap closing. No reviewed work places a memory engine beside a frozen small model on a consumer laptop and reports what fraction of the gap to a cloud model it closes, separately for knowledge-bound and reasoning-bound tasks, at what cost in tokens and latency, and against both a plain vector database and a cloud model. Narayan et al. (2025) came nearest, but their on-device model depended on the cloud throughout. This intersection defines the gap that the study addressed.

2.6 Conceptual Framework

Figure 2.7 presents the conceptual framework that structured the study, as a flow from input, through processing, to output. It follows two example questions, one knowledge-bound and one reasoning-bound, from the input to their answers in the output. The input variables are the person's question, which is either knowledge-bound or reasoning-bound, and the person's vault of notes, documents, and code. The processing variables are the five steps of the engine, Decide, Fit, Answer, Check, and Learn, which together form the write policy, the retrieval policy, and the token budget. The language model is held constant, with its weights frozen, so that every measured gain can be attributed to the memory engine. The output variables are the answer with its sources, the decision line that states what the engine did, and the notes the person agrees to keep, which return to the vault. Every answer was measured for whether it was right, the fraction of the gap to the cloud model closed, the tokens read, and the seconds taken. Task type moderates the relationship: the framework rests on the finding that retrieval closes the gap on rare facts (Kandpal et al., 2023; Mallen et al., 2023) and that irrelevant text harms reasoning (Shi et al., 2023), so the study predicted substantial gap closing on knowledge-bound questions and little on reasoning-bound questions. The framework guided the design in Chapter 4 and the experiments in Chapter 5.

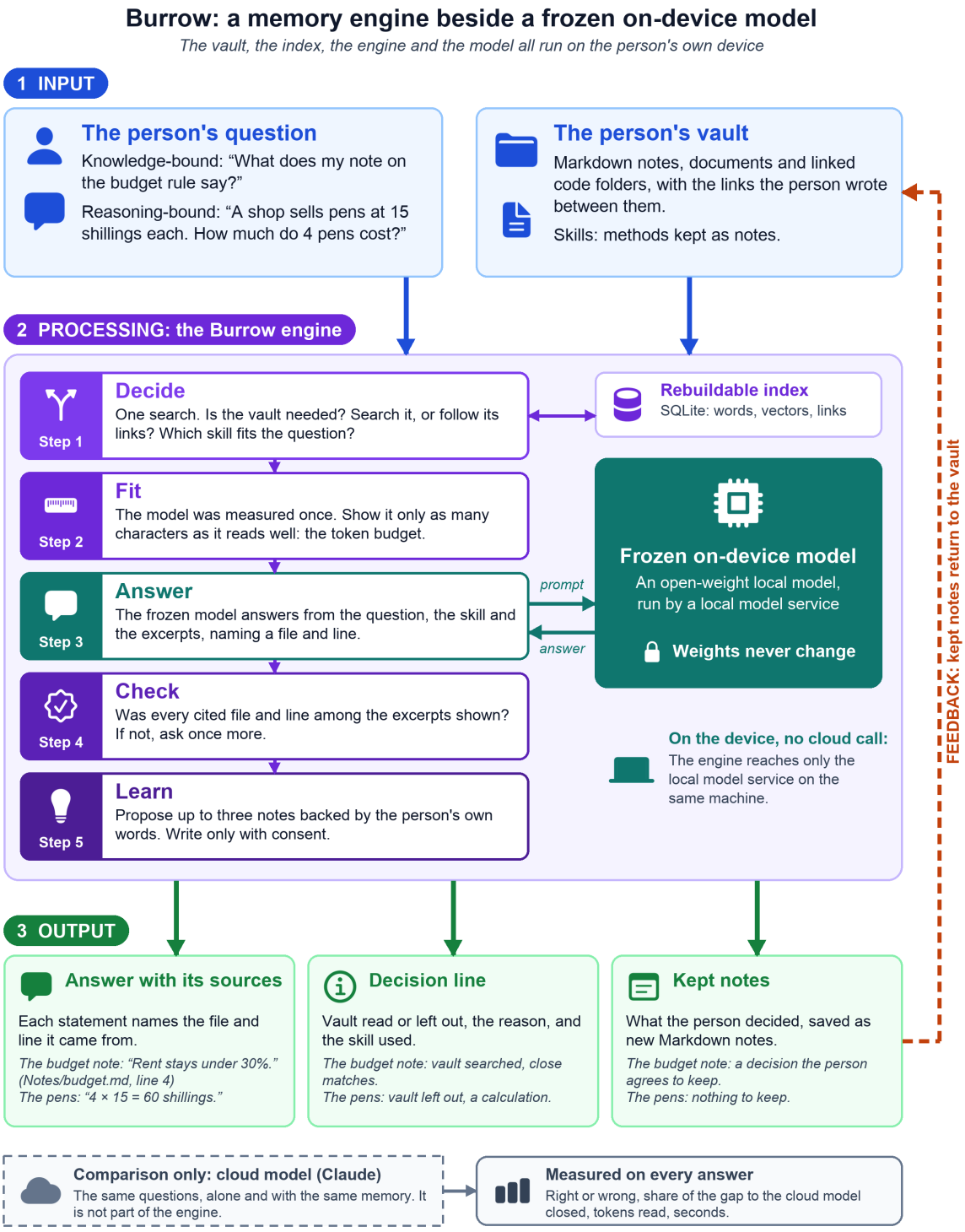


Figure 2.7: Conceptual framework of the study, drawn by the author

Chapter 3: Methodology

3.1 Introduction

This chapter describes how the study was carried out. It states the research paradigm that guided the work, the development approach that was followed, the analysis of what the system must do, the design of both the system and the experiments that evaluated it, the tools and techniques that were used, and the deliverables the study produced. The system was analysed and designed with Object-Oriented Analysis and Design (OOAD). Throughout the chapter, the guiding principle is that every claim about the engine was tested as a controlled experiment.

3.2 Research Paradigm

The study adopted a pragmatist research philosophy, combining design science research with controlled experimentation. Design science research is appropriate because the study created and evaluated a new artefact, the memory engine, and the discipline's guidelines require that such an artefact be evaluated rigorously against explicit measures of utility (Hevner et al., 2004). The study followed the design science research methodology of Peffers et al. (2007), proceeding through problem identification, definition of solution objectives, design and development, demonstration, evaluation, and communication. The evaluation took the form of controlled experiments organised as a research pipeline of four stages, described in the subsections that follow.

3.2.1 Data Acquisition

Two kinds of data were acquired, so that knowledge-bound and reasoning-bound performance could be separated. The knowledge data was a curated vault on three subjects that an on-device model cannot know well: Kenyan data protection law, taken from the Data Protection Act and its three sets of 2021 regulations (Data Protection Act, 2019); software documentation dated 2026, namely the release notes of SQLite, Python 3.15, Ollama, and the Model Context Protocol; and a private software project, the source code and decision record of Burrow itself at an earlier version. The reasoning data was 50 problems from GSM8K, a public benchmark of grade-school mathematics problems (Cobbe et al., 2021). All of the documents are public or belong to the author, and none contains personal data about any person. Each record is a question, its answer key, and, for a knowledge question, the passage that supports the key, as Table 3.1 shows.

Table 3.1: Two records of the evaluation data

Kind	Question	Answer key
Knowledge, one fact	Under Kenya's Data Protection Act, 2019, within how many hours of becoming aware of a personal data breach that poses a real risk of harm must a data controller notify the Data Commissioner?	seventy-two or 72
Knowledge, several facts	Across SQLite 3.53.0 and 3.54.0 (both released in 2026), which three new options were added to the sqlite3_limit() interface?	SQLITE_LIMIT_PARSER_DEPTH ; SQLITE_LIMIT_SCHEMA; SQLITE_LIMIT_TRIGGER_STEP S

3.2.2 Data Preparation

The documents of the three subjects were converted to Markdown files and placed in one vault, so that the documents of two subjects stood as distractors for the third. The engine then indexed the vault before any evaluation began. For each subject, 25 single-fact questions and 15 multi-fact questions were written from the documents by Claude Code agents, and a script checked every key against the text of the vault and rejected any key that appeared in its own question. The reasoning problems were drawn once from the GSM8K test split with a fixed random seed. The vault was built from the source documents only, never from the questions or their keys, so that the experiment measured the engine's ability to surface relevant knowledge rather than its ability to look up a planted answer. The same questions were reused in every set-up, so that every comparison was paired.

3.2.3 Experimental Conditions

Six set-ups were run on every evaluation question. The floor was the on-device model alone, the ceiling was the cloud model alone, and the treatment was the on-device model with the memory engine. No model was trained or fine-tuned; the weights stayed frozen in every set-up, because the engine works beside the model and does not change it. The full design of these conditions is given in Section 3.5.2.

3.2.4 Measurement and Validation

Every question in every set-up logged the model used, the tokens read and written, the seconds spent before the model was called, the seconds the model took, and the resulting answer, so that every experiment could be examined from its logs. The central quantity of the

study, the fraction of the capability gap closed, was computed from the set-ups as defined in Section 3.5.2. Validity was protected by the paired design, an evaluation plan written before any scored run, a program frozen before the first scored run, and the measures described in Section 3.5.6.

3.3 Development Approach

The system was developed by incremental prototyping, in small working stages, each of which was checked before the next was begun. Iterative and incremental development grows a system in small steps in which the result of each step informs the next (Larman & Basili, 2003), and design science asks for the same cycle of building and evaluating (Hevner et al., 2004). This approach suited the study for two reasons. First, the requirements changed: the Sprint II review of 5 November 2026 asked for something a person can see and use, and an incremental plan could take that in without losing the work already done. Second, the engine is a chain of separable steps, so its parts could be built in turn and switched off when it was evaluated. The system was analysed and designed with OOAD, because the memory is naturally a set of objects that hold their own data and behaviour, and Chapter 4 therefore presents use case, sequence, activity, and class diagrams drawn in the Unified Modeling Language.

Development proceeded in the five increments shown in Figure 3.1. The first increment built a memory engine and an evaluation harness in Python under the working name CAMR, from the approved proposal, and ended at the Sprint II review. The second increment rebuilt the memory in Swift as Burrow, with a vault of plain Markdown files, a rebuildable SQLite index, retrieval by words and by meaning, and a walk over the links between files. The third increment added the surfaces through which a person and other tools use the memory: a native desktop application, a terminal chat, and a Model Context Protocol server. The fourth increment froze the retrieval and tested it on 101 real bug fixes from public repositories and on three held-back synthetic projects worked by simulated developer agents; memory did not measurably help the small model write working code, which showed that finding the right text was not enough. The fifth increment built the engine around the model, namely the Decide, Fit, Check, and Learn steps and the skills, wrote the evaluation harness of Section 3.5, froze the program, and ran the evaluation.

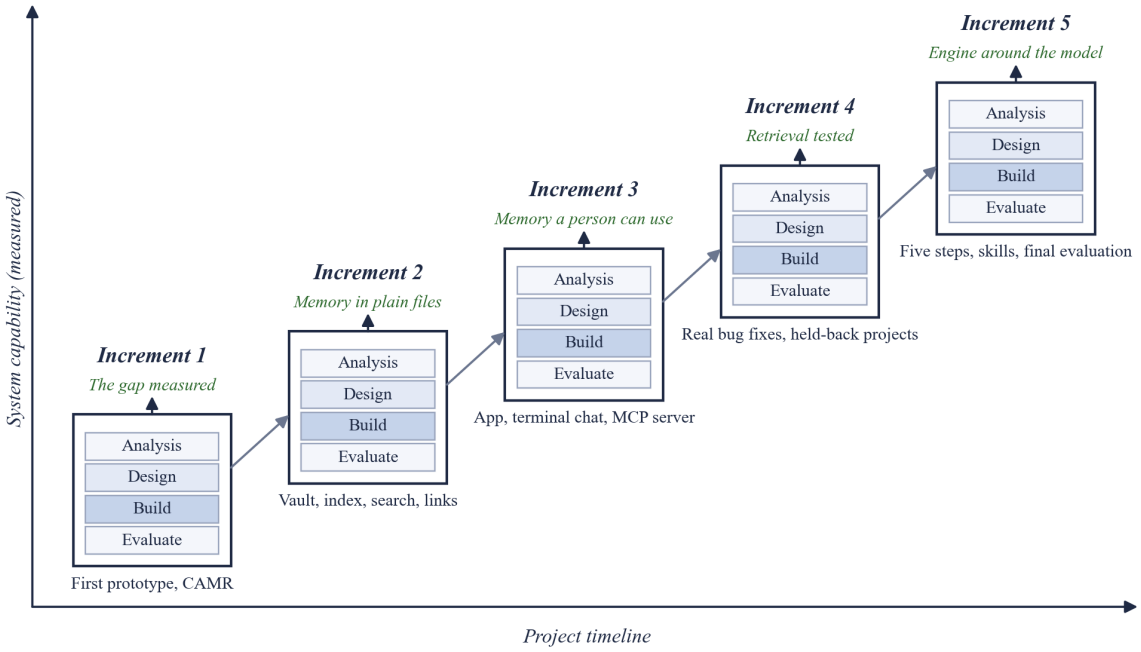


Figure 3.1: Incremental development approach for the Burrow engine, drawn by the author

3.4 System Analysis

The requirements for the engine were formulated by working backwards from the study objectives, the measurements the evaluation demands, and the Sprint II review, so that every capability maps to something the experiments or the person need. In functional terms, the engine must keep a vault of the person's notes, documents, and code with a rebuildable index, decide for each question whether the vault is needed and how to read it, retrieve and rank passages by words, meaning, and links, fit them to a token budget measured for the model, have the model answer with its sources, propose notes that are written only with the person's agreement, serve other tools through a standard protocol, and log the tokens, latency, and answer of every question so that each experiment can be examined. In non-functional terms, it must run entirely on the target consumer laptop with no dedicated graphics hardware and only the cloud reference model reached over a network, add a small fraction of the time of an answer, keep the vault as plain files beside a single local index with no server process, and retain all data on the device in every configuration except those that use the cloud model.

3.5 System Design

This section describes the design of the memory engine and, with equal weight, the design of the experiments that evaluated it.

3.5.1 Architectural Overview

The engine consists of five steps arranged around a frozen local language model. Decide runs one search and judges whether the vault is needed, whether to search it or follow its links, and which skill fits the question. Fit limits what the model is shown to the amount measured for that model. Answer has the frozen model write the answer from the question, the skill, and the excerpts, naming a file and line for each statement. Check compares every cited place with what the model was shown and asks once more if it was not. Learn proposes notes for the person to keep. The store is a vault of plain Markdown files with a single embedded SQLite index beside it, which holds a word index, embedding vectors, and the links between files, and which can be rebuilt from the files at any time. The detailed component design is presented in Chapter 4.

3.5.2 Experimental Design

The central quantity of the study is the fraction of the capability gap closed, and the experiment was designed around measuring it cleanly. Six set-ups were run on every question: the on-device model alone, which is the floor; the on-device model with plain retrieval; the on-device model with a vector database baseline built on Chroma; the on-device model with the engine, which is the treatment; the cloud model alone, which is the ceiling; and the cloud model with the memory prepared by the engine. The first four were run with both on-device models, at a temperature of zero. Plain retrieval and the Chroma baseline used the same vault, the same embedding model, and the same limit of 12,000 characters. The engine showed Gemma 4 12B up to 17,417 characters, the budget its fit produced, and Qwen3.5 4B up to 12,000, and the cloud model with the engine was shown up to 24,000. For each task type, the gap closed was computed as shown in Equation 3.1, where S denotes the accuracy of a set-up on that task type.

$$Gap\ closed\ = \frac{S_{\text{treatment}} - S_{\text{floor}}}{S_{\text{ceiling}} - S_{\text{floor}}}$$

Equation 3.1: Fraction of the capability gap closed

A value of zero means the engine added nothing, and a value of one means the on-device model with the engine matched the cloud model. Where the cloud model was not ahead of the on-device model alone, the measure is not defined. A rule added after the evaluation plan was written leaves the fraction out of Table 5.6 when the ceiling is less than ten percentage points above the floor, because one answer then moves it widely.

3.5.3 Datasets and Task Types

The 120 knowledge questions, 40 on each of the three subjects, were the knowledge-bound task type. Each subject held 25 single-fact questions and 15 multi-fact questions, and a multi-fact question needs two passages from two different files. These questions are the case where a small model fails most severely, because it cannot have memorised the material. The 50 GSM8K problems were the reasoning-bound control, where the required ability is reasoning rather than recall and where memory was predicted to help little (Cobbe et al., 2021). The question for this control was whether the engine would leave the vault out and protect the answer.

3.5.4 Evaluation Metrics

A knowledge answer was right when it contained every required fact of its key, and a reasoning answer was right when its last number equalled the key. Accuracy was the share of right answers in a set-up, and the gap closed was computed from it per task type using Equation 3.1. Retrieval was also judged apart from answering, by recording whether every file that holds the answer was among the excerpts shown. Efficiency was reported as the tokens read, the tokens written, the seconds spent before the model was called, and the seconds the model took, all measured on the target device. Paired differences between set-ups were tested with an exact two-sided sign test, because each answer is simply right or wrong.

3.5.5 Ablation Procedure

Parts of the engine were evaluated by switching them off and measuring the change on the same questions, holding everything else fixed. The full engine on the main model was the reference. One run removed the skills and another removed the citation check, and each was reported beside the full engine. The token budget was varied separately on the 120 knowledge questions, with the amount shown fixed at 2,000, 4,000, 8,000, and 32,000 characters beside the fitted 17,417, which answers the fifth specific objective. Decide and Fit were not switched off on their own.

3.5.6 Threats to Validity

Six threats were addressed. Answer leakage was prevented by building the vault from the source documents only (Section 3.2.2). Prompt sensitivity was limited by one fixed prompt for each set-up. Sampling error was addressed by paired questions and an exact sign test. Timing noise was addressed by reporting medians on one machine, although each question

was run once. Ceiling instability was addressed by pinning and logging one version of the cloud model. Tuning bias was addressed by setting the thresholds of the engine on separate development questions, although several concern the same subjects as scored questions, while the scored questions were not read during tuning and played the part of a held-out test set.

3.6 Tools and Techniques

This section describes the tools that were used to build the engine and the harness, and the role each played. Their exact versions are given in Section 5.2.2.

3.6.1 Swift

The engine, the command-line tool, the terminal chat, the protocol server, and the desktop application were written in Swift, the native language of the target platform, so that the engine runs as one compiled program with no interpreter.

3.6.2 SQLite with FTS5

The index is SQLite, one database file with no server process. The word index uses the FTS5 full-text extension (SQLite, 2026), and the embedding vectors are stored in the same database with a memory-mapped copy beside it for speed.

3.6.3 Ollama and Local Models

Local models were served through Ollama, which runs quantised open-weight models on consumer hardware (Ollama, 2026). The on-device models were Gemma 4 12B and Qwen3.5 4B (Gemma Team, 2026; Qwen Team, 2026).

3.6.4 Hosted Inference API

The cloud reference model, Claude (Anthropic, 2026c), was reached through its hosted API and pinned to a single version for the whole study, so that the ceiling remained stable across all runs.

3.6.5 Embedding Model

Embedding vectors were produced on the laptop by Qwen3 Embedding 0.6B (Y. Zhang et al., 2025). The engine and the baselines used the same model.

3.6.6 Chroma and LangChain

Chroma, an open-source vector database (Chroma, 2026), was run locally to build the baseline of Section 3.5.2. LangChain, a framework for applications around language models (LangChain, 2026), was supported through an adapter that exposes Burrow as a retriever.

3.6.7 Model Context Protocol

The Model Context Protocol, an open standard between AI applications and data sources (Model Context Protocol, 2026), was used so that outside models and tools can reach the same memory.

3.6.8 Auxiliary Tools

The evaluation harness, the scripts that check the answer keys, and the charts were written in Python. Claude Code wrote the knowledge questions under the checks of Section 3.2.2. Version control used Git and GitHub, and the state of the code at each evaluation was marked with a tag, so that a result can be traced to the code that produced it.

3.7 Deliverables

The study produced the following deliverables, each described in its own subsection.

3.7.1 The Burrow Memory Engine

A Swift package that implements the vault, the index, and the five steps described in Section 3.5.1, with a command-line tool named burrow, running entirely on the target consumer laptop.

3.7.2 The Desktop Application, Terminal Chat, and Protocol Server

A native desktop application in which a person writes notes, follows their links, and asks questions of the vault, a terminal chat, and a Model Context Protocol server that gives outside models and tools the same memory.

3.7.3 The Evaluation Harness

The question sets, set-up runners, scorers, and logging needed to repeat every experiment in the study from the command line.

3.7.4 Experimental Results

The measured gap closed per task type, the ablation and budget results, and the cost of each answer in tokens and seconds, together with the logs that support them.

3.7.5 System Documentation

This report, including the design, the experiments, and the findings, with the instructions, configuration files, and prompt templates needed to reproduce the results.

Chapter 4: System Analysis and Design

4.1 Introduction

This chapter presents the analysis and design of Burrow, the memory engine of this study, and addresses the second specific objective. The system was analysed and designed with Object-Oriented Analysis and Design (OOAD), as justified in Section 3.3, and the diagrams use the Unified Modeling Language (UML). Section 4.2 gives the requirements of the system. Section 4.3 presents the analysis diagrams, which are a use case diagram, two sequence diagrams, an activity diagram, and a class diagram. Section 4.4 presents the design diagrams, which are the system architecture, the engine's steps, the logical database schema, the wireframes, and a deployment diagram.

4.2 System Requirements

The system requirements identified for this project are presented in the following subsections. They were worked backwards from the specific objectives in Section 1.3 and from the Sprint II review of 5 November 2026. Each requirement has an identifier, a priority, the objective it serves, and the test case of Section 5.6 that verifies it. A requirement is marked Must when an objective depends on it and Should when it was added to strengthen the engine. No limit on response time was set in advance, because no measurement existed to justify one.

4.2.1 Functional Requirements

Table 4.1 lists the functional requirements. FR-01 to FR-03 cover the memory, FR-04 to FR-09 are the five steps of the engine, and FR-10 opens the engine to outside models.

Table 4.1: Functional requirements

ID	Requirement	Priority	Objective	Verified by
FR-01	The engine shall index a vault of Markdown notes, documents, and linked code folders on the device, and later re-index only the files that changed.	Must	iii	TC-01
FR-02	The engine shall rank the pieces of the vault by word match and by meaning, and return excerpts with their file and line numbers within a character budget.	Must	ii	TC-02
FR-03	The engine shall follow the links between files for up to two hops from the best matches.	Must	ii	TC-03
FR-04	Decide: the engine shall judge from one search whether a question needs the vault, and whether to search or to follow links, and state the reason in words.	Must	ii	TC-04, TC-05
FR-05	Decide: the engine shall hand the model at most one skill, chosen by how closely it fits the question or else by the kind of material found.	Should	ii	TC-06
FR-06	Fit: the engine shall measure a local model once and set from that measurement how many characters the model is shown.	Must	ii, v	TC-07
FR-07	Answer: the engine shall have a local model answer from the excerpts, naming a file and line for each statement.	Must	iii	TC-08
FR-08	Check: the engine shall check every cited file and line against the excerpts shown, ask once more if a citation was not shown, and report any left unverified, on the command burrow ask --mode engine.	Should	ii	TC-09
FR-09	Learn: the engine shall propose at most three notes from a conversation, each backed by the person's own words, and in the terminal chat write a note only when the person accepts it.	Must	ii	TC-10
FR-10	The engine shall give the same decision, skill, and excerpts to an outside model through a command and through a Model Context Protocol (MCP) tool, without calling a chat model itself.	Should	iii	TC-11

4.2.2 Non-Functional Requirements

Table 4.2 lists the non-functional requirements. They cover privacy, durability, safety, cost, footprint, and portability, which are the qualities that matter for a memory kept on a person's own device. NFR-04 requires that cost be measured on every answer.

Table 4.2: Non-functional requirements

ID	Requirement	Priority	Objective	Verified by
NFR-01	Privacy: the engine shall send notes and code only to the Ollama service on the same machine, at the loopback address 127.0.0.1.	Must	iii	TC-13
NFR-02	Durability: the person's files shall be the only lasting record, and the index shall be rebuilt from them if it is deleted.	Must	iii	TC-12
NFR-03	Safety: text from the vault shall be given to a model as material to read, with a rule that no instruction written inside it is followed.	Should	iii	TC-14
NFR-04	Cost: for every answer the engine shall report the tokens read, the tokens written, the seconds before the model was called, and the seconds the model took.	Must	iv	TC-15
NFR-05	Footprint: the index shall be one embedded database file inside the vault's .burrow folder, with no database server process. A rebuildable copy of its vectors and two small settings files sit beside it.	Must	iii	TC-16
NFR-06	Portability: the engine shall build as one Swift package with no third-party dependency, and its tests shall run with one command.	Should	iii	TC-17

4.2.3 Data Requirements

Table 4.3 lists the data requirements. They describe what the engine reads, what it stores, what it is allowed to write, and how the evaluation data is kept apart from the memory.

Table 4.3: Data requirements

ID	Requirement	Priority	Objective	Verified by
DR-01	The input shall be a folder of plain Markdown files and linked code folders. The engine shall read these files and shall not change them.	Must	iii	TC-18
DR-02	For each piece of a file the index shall store its file, its first and last line, its text, and at most one embedding vector.	Must	iii	TC-19
DR-03	In the chat and the app a note shall be written only when the person accepts it, as a readable Markdown file in the Memory folder. No write shall overwrite a note. Other writes are limited to the starter skills on request, accepted file changes, and notes added by burrow note, burrow learn, or an MCP client.	Must	ii	TC-10
DR-04	The evaluation questions and their answer keys shall be kept outside the vault, and the vault shall be built from the source documents only.	Must	iv	TC-20

4.2.4 Interface and Integration Requirements

Table 4.4 lists the interface and integration requirements. They describe how a person, the local model service, and outside tools meet the engine, and what happens when the model service fails.

Table 4.4: Interface and integration requirements

ID	Requirement	Priority	Objective	Verified by
IR-01	Under each answer the application shall show what the engine decided, the skill it used, and the files it read, and shall open a file at the cited line.	Must	Sprint II	TC-08
IR-02	If the Ollama service cannot embed a question, the search shall continue on words alone. If it cannot answer, the failure shall be reported and no answer shown.	Must	iii	TC-21
IR-03	The MCP server shall exchange JSON-RPC messages over standard input and output, and shall offer its tools only after the client has initialised the session.	Should	iii	TC-11
IR-04	The vault shall stay a folder that Obsidian can open: plain Markdown with links written between double square brackets.	Should	iii	TC-22
IR-05	The system shall be offered from a public landing page, with the terminal version installed by one command and the desktop application downloaded by one click, both from the project's published releases.	Should	Sprint II	TC-23

4.3 System Analysis Diagrams

This section presents the diagrams used to analyse the actors, the behaviour, and the structure of the system. Each diagram is introduced, shown, and then interpreted.

4.3.1 Use Case Diagram

The use case diagram gives the widest view of the system, namely who uses Burrow and for what. It was chosen because the engine serves three kinds of actor, and it addresses requirements FR-04 to FR-10. Figure 4.1 shows the boundary of the system and its actors: the person who owns the vault, the local model served by Ollama, and an outside model or tool that brings its own model.

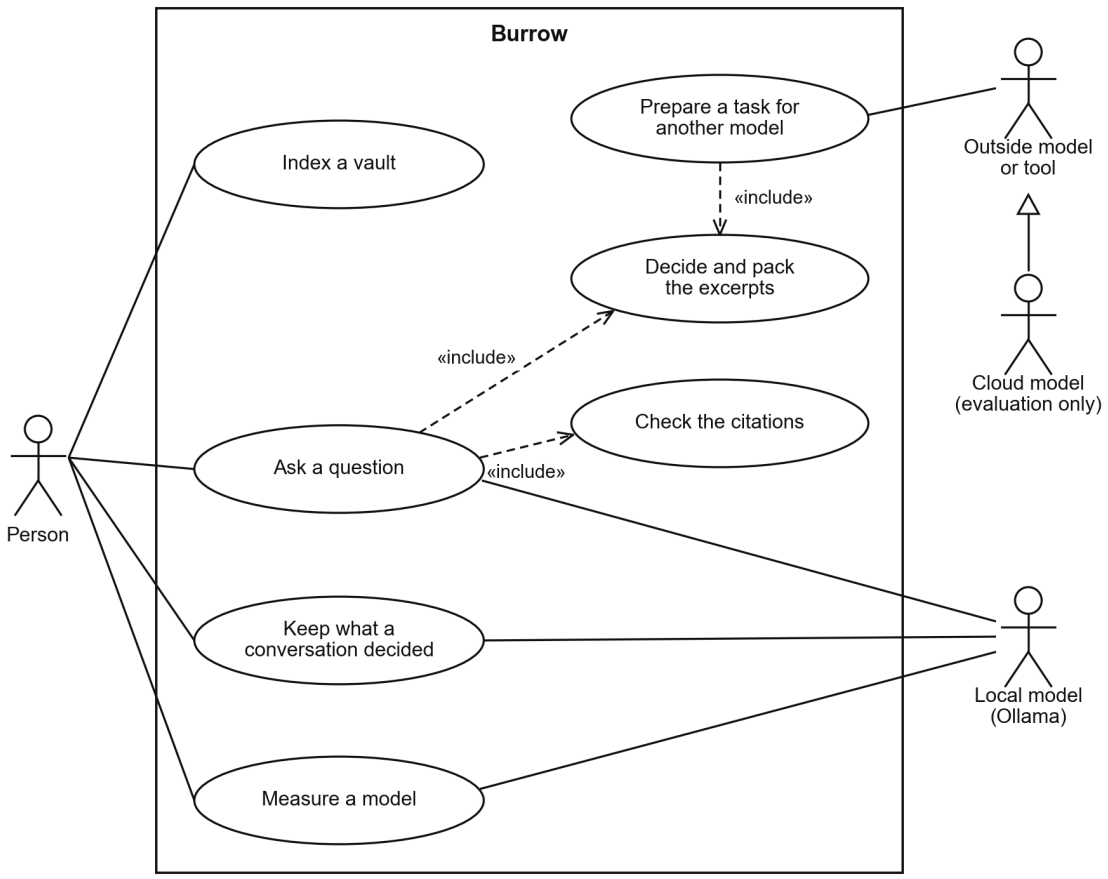


Figure 4.1: Use case diagram of Burrow

Asking a question and preparing a task for another model both include the same use case, deciding and packing the excerpts, so the engine's judgement is shared by every way of using it. Only the path of the local model includes checking the citations, because only there does the engine see the answer. Table 4.5 describes the use case that carries the study, "Ask a question", which exercises four of the five steps.

Table 4.5: Use case description: Ask a question

Item	Description
Actors	Person (primary), local model (supporting)
Precondition	The vault has been indexed. Ollama is running with a chat model.
Main flow	1. The person asks a question. 2. The engine searches once and decides: leave the vault out, search, or follow links. 3. The engine picks a skill and packs the excerpts within the model's budget. 4. The local model answers. 5. The engine checks each file and line cited. 6. The person receives the answer, the decision line, and the sources.
Alternative flows	At step 2 nothing in the vault is close, so the model answers without excerpts. At step 5, on the command line, a citation was not among the excerpts, so the model is asked once more and the better grounded answer is kept.
Exceptions	Ollama cannot embed the question, so ranking continues on words alone. Ollama cannot answer, so the failure is reported and no answer is shown.
Postcondition	No file in the vault has changed. Any citation that could not be verified is named.

4.3.2 Sequence Diagrams

A sequence diagram shows the order of the messages between the parts of the system. It was chosen because the five steps must run in a fixed order, and the first diagram addresses FR-04 to FR-08. Figure 4.2 shows the messages exchanged when a person asks a question through the engine.

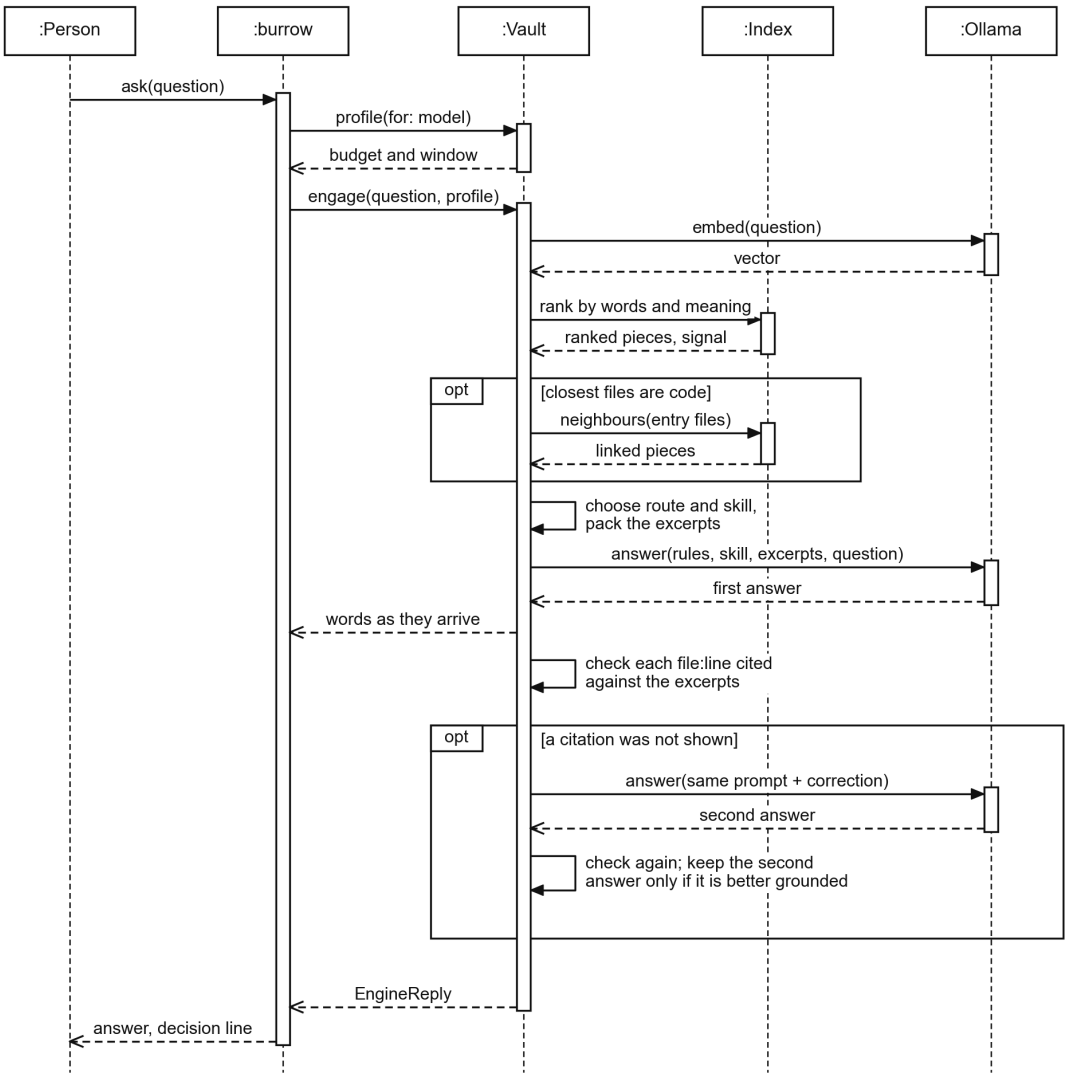


Figure 4.2: Sequence diagram of a question asked through the engine

The model profile is fetched first, so the budget is known before any search begins. The local model is called once in the usual case and twice at most, the second time with the wrongly cited places named. Figure 4.3 shows the second way in, which addresses FR-10: an outside model using Burrow through the MCP tool vault_prepare.

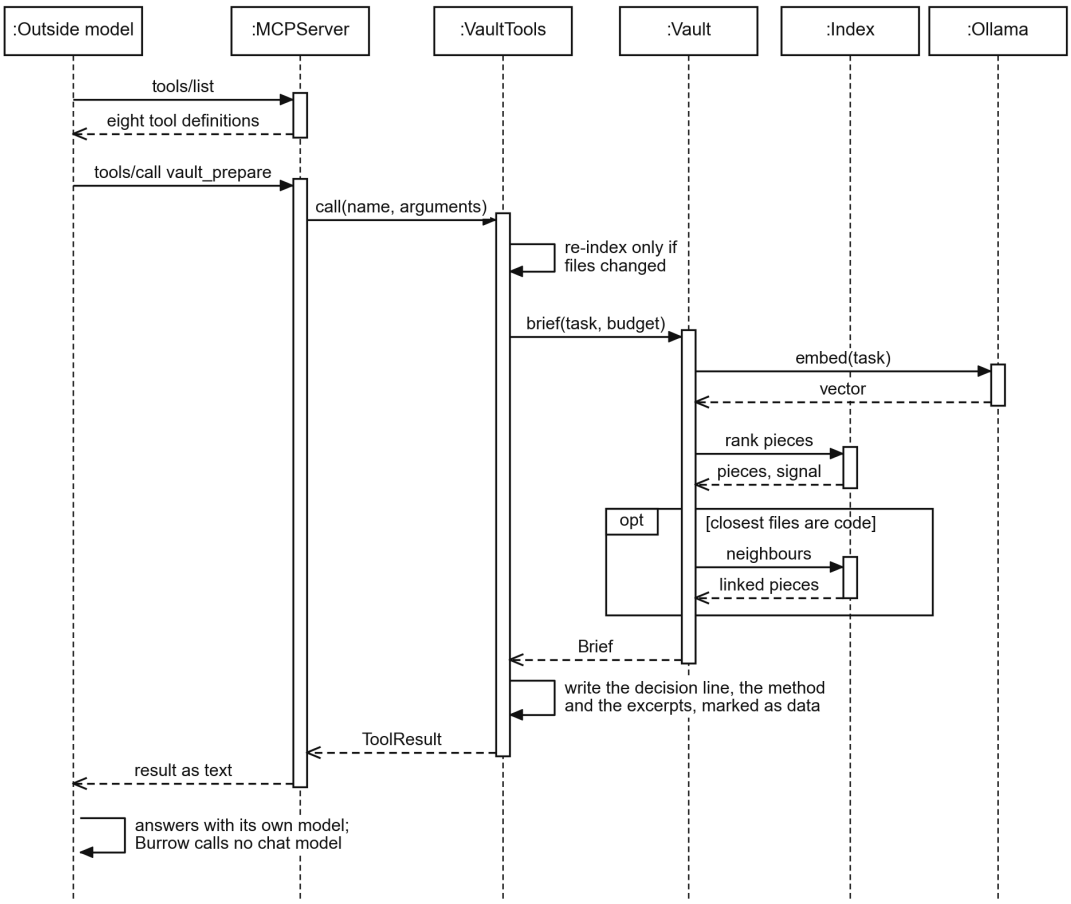


Figure 4.3: Sequence diagram of an outside model preparing a task through MCP

The only model Burrow calls on this path is the embedding model. The outside model receives the decision, a skill, and the excerpts, and writes the answer itself. Burrow therefore never sees that answer and cannot check its citations.

4.3.3 Activity Diagram

The activity diagram shows the decisions of the engine's loop and who makes each one. It was chosen because the study depends on separating what the engine decides from what the model writes, and it addresses FR-04 to FR-09. Figure 4.4 follows the loop from a question to a kept note, in three swimlanes for the person, the engine, and the local model.

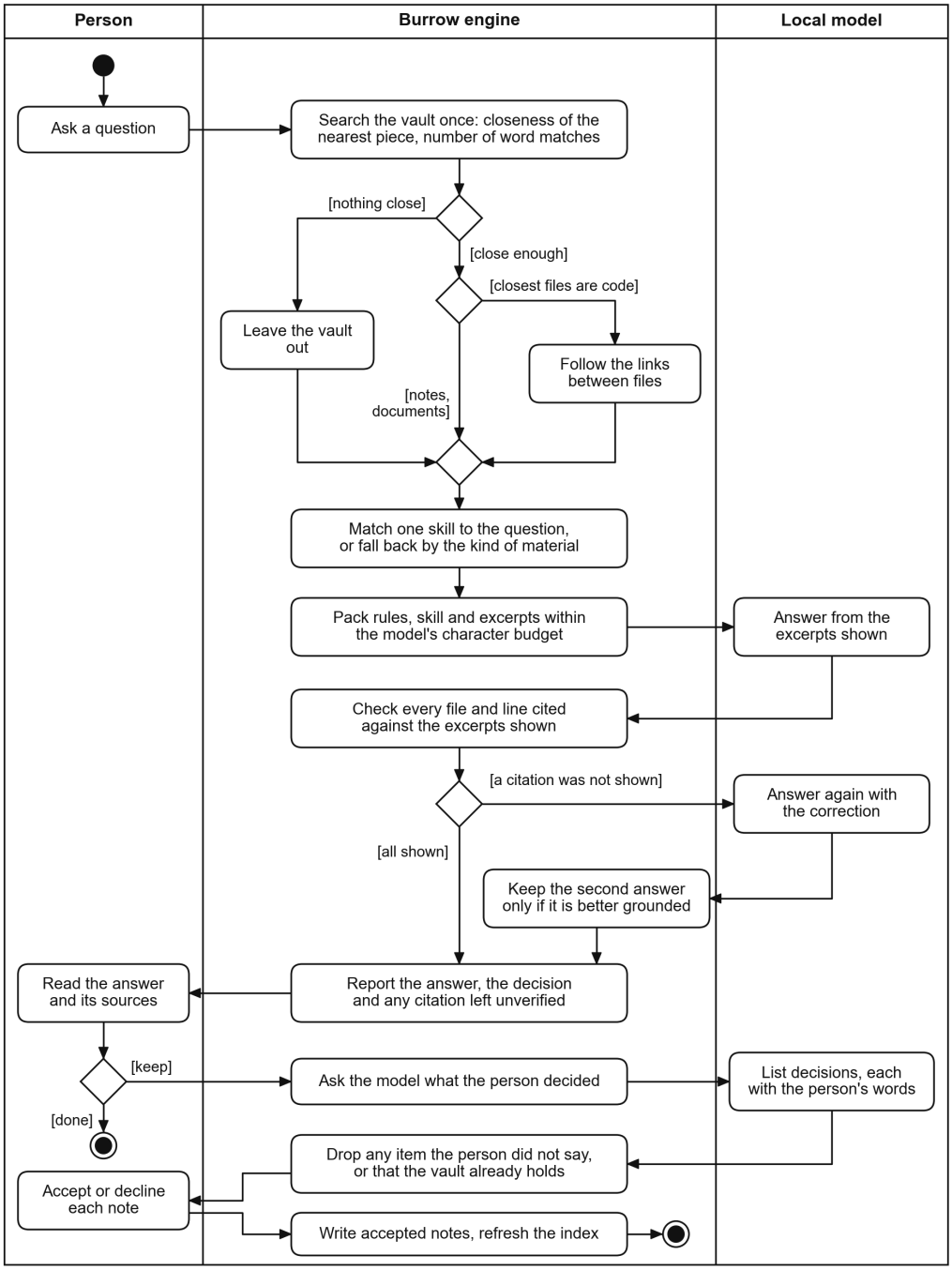


Figure 4.4: Activity diagram of the engine's loop

Every choice about what the model reads is made in the engine lane, before the model is called. Each time the model returns something, the engine examines it before it is used. The last decision belongs to the person, who accepts or declines each proposed note.

4.3.4 Class Diagram

The class diagram shows the types of the engine and how they depend on one another. It was chosen because it maps directly to the Swift code and guided how the library was divided. Figure 4.5 shows the main types as they were built.

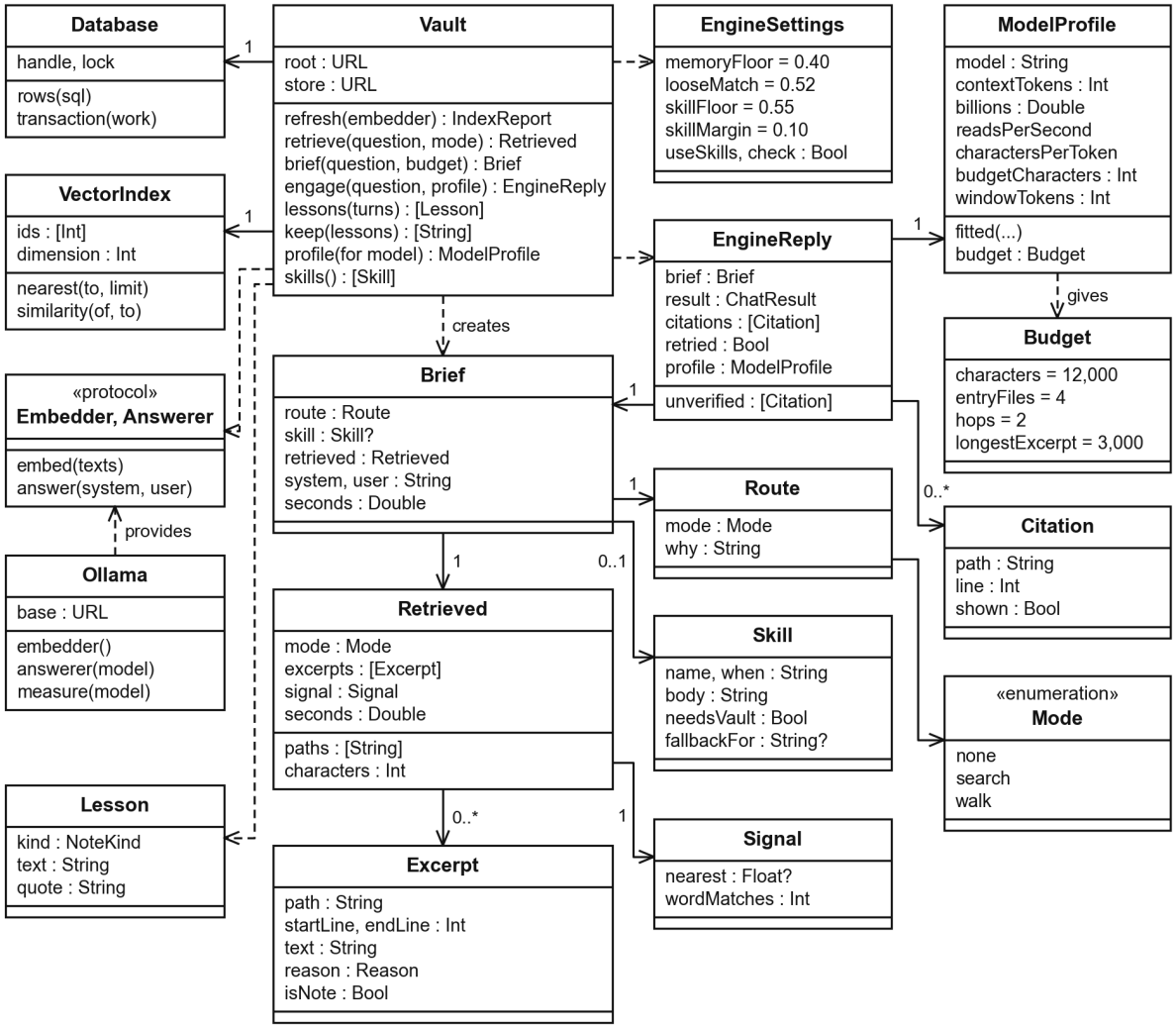


Figure 4.5: Class diagram of the engine

The Vault type is the single entry point: brief decides and packs, engage adds the answer and the check, and lessons and keep are the two halves of Learn. A Brief contains no answer, which is why it can be handed to an outside model. Embedder and Answerer are protocols, so the tests run the whole loop with a scripted model in place of a real one.

4.4 System Design Diagrams

This section presents the diagrams that show how the system was organised to satisfy the analysed requirements.

4.4.1 System Architecture

The architecture diagram shows the components of Burrow and which component depends on which. It was chosen to show that all memory logic sits in one library that every surface uses,

and it addresses FR-10 and NFR-06. Figure 4.6 shows the components in three layers: the surfaces that a person or a tool touches, a shared set of tools, and the core.

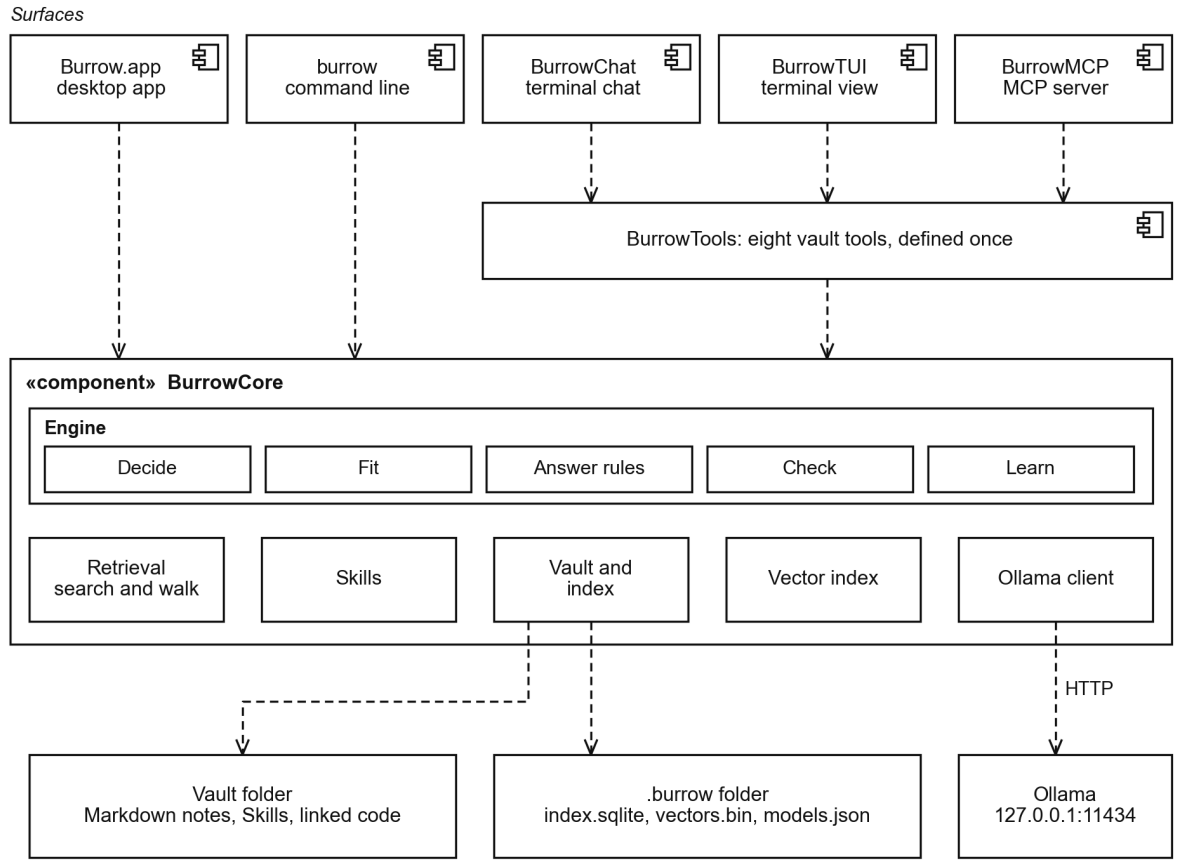


Figure 4.6: Architecture of Burrow

All memory logic is in BurrowCore, and every surface depends on it, so a change to the engine reaches the application, the terminal chat, and the MCP server at once. The Ollama client is the only part that leaves the process. Not every surface runs every step: the check with retry runs only on the command line, the app warns about an unverified citation, and the terminal chat and the MCP server use a fixed budget of 12,000 characters and not the fitted one.

4.4.2 Engine Steps and Retrieval Design

Objective ii names three mechanisms: a write policy, a retrieval policy, and a token budgeting mechanism. Figure 4.7 shows the five steps of the engine, what each reads and writes, and how the steps map to those mechanisms.

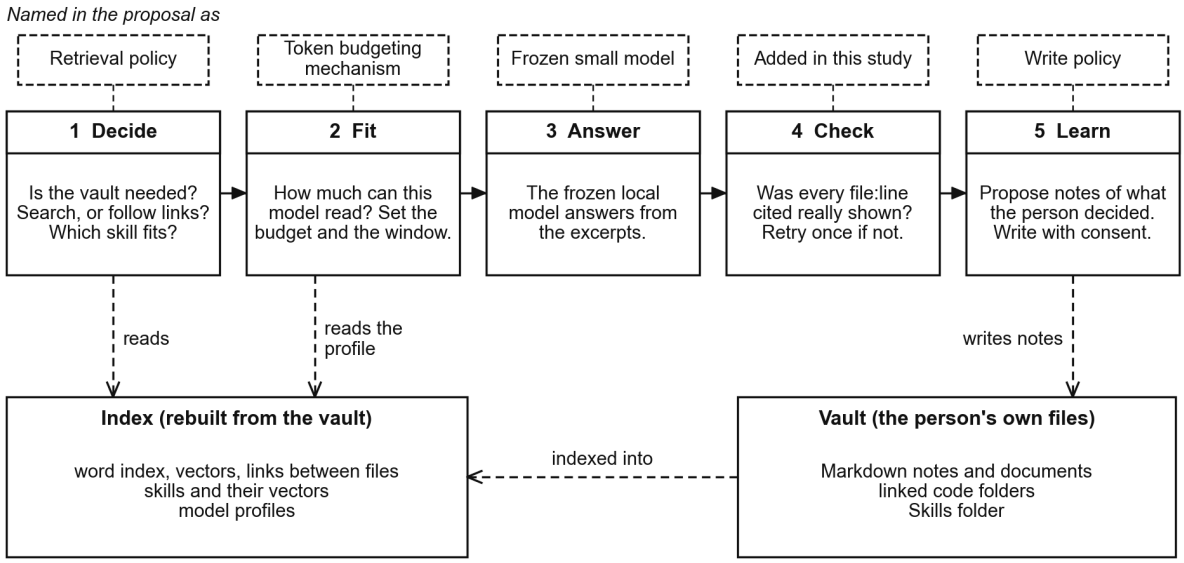


Figure 4.7: The five steps of the engine and the mechanisms of the proposal

Learn is the write policy, Decide and the retrieval behind it are the retrieval policy, and Fit is the token budgeting mechanism. Check was added in this study. The recency and importance scores of the proposal were not built, and ranking is by words, meaning, and links, as Section 5.4.7 records.

Retrieval ranks each piece twice, by word match with BM25 (Robertson & Zaragoza, 2009) and by meaning. Meaning is the cosine between the vector of the question, q, and the vector of a piece, v, as in Equation 4.1.

$$cos(q, v) = \frac{q \cdot v}{\|q\| \, \|v\|}$$

Equation 4.1: Cosine similarity between a question and a piece

The two rankings are merged by reciprocal rank fusion (Cormack et al., 2009), as in Equation 4.2, where the rank of piece p in ranking r counts from 1 and w(p) lowers the weight of test files and of the output of past runs. Pieces holding an exact name from the question, or from a file the question names, are placed before this ranking.

$$score(p) = w(p) \sum_{r \in \{words, meaning\}} \frac{1}{60 + rank_r(p)}$$

Equation 4.2: Merged score of a piece

When links are followed, a neighbouring file n is ranked by Equation 4.3, where b(n) is its word match scaled so that the best file scores 1, h is the hop number, and k(n) depends on the kind of link.

$$s(n) = w(n)\left(\frac{b(n)}{h} + k(n)\right)$$

Equation 4.3: Score of a neighbouring file during a walk

4.4.3 Logical Database Schema

The index is the only database in the system, and it holds no original data. The schema was chosen as a design view because its tables decide what can be searched by words, by meaning, and by links, and it addresses DR-02 and NFR-02. Figure 4.8 shows its logical schema, which is one SQLite database.

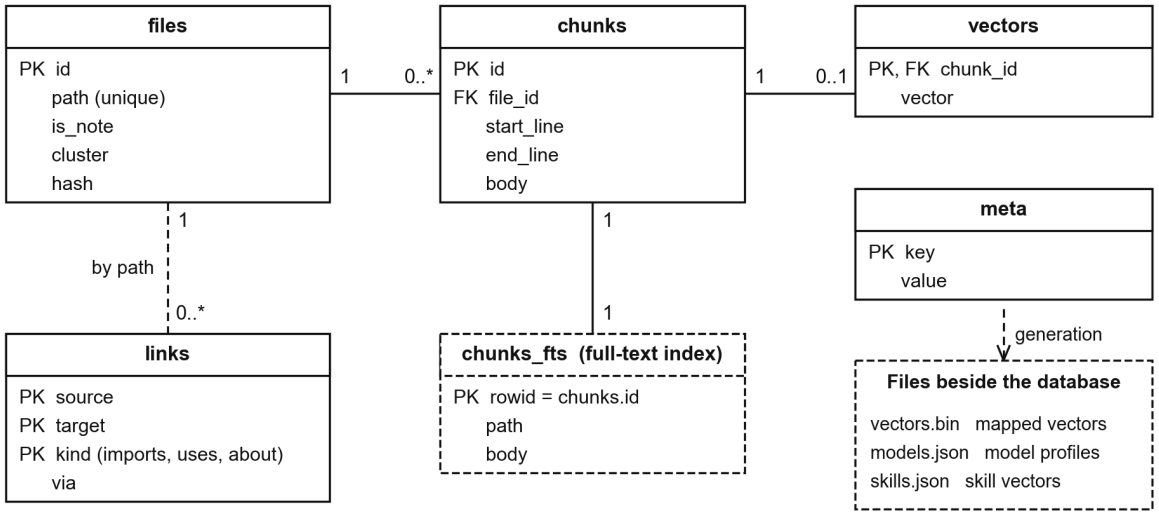


Figure 4.8: Logical schema of the index

A file is cut into pieces stored as chunks, and a piece has at most one vector, because the index also works on words alone. The chunks_fts table is the full-text index, and the links table holds the links between files. Nothing in the schema is a source of truth: the whole index, with the memory-mapped copy of its vectors and two small settings files beside it, can be rebuilt from the Markdown files, which satisfies NFR-02.

4.4.4 Wireframes

The wireframes planned where the engine's decisions would be shown to the person, which addresses IR-01 and the Sprint II request for something a person can see and use. Figure 4.9 is the main window of the desktop application with the Ask panel open.

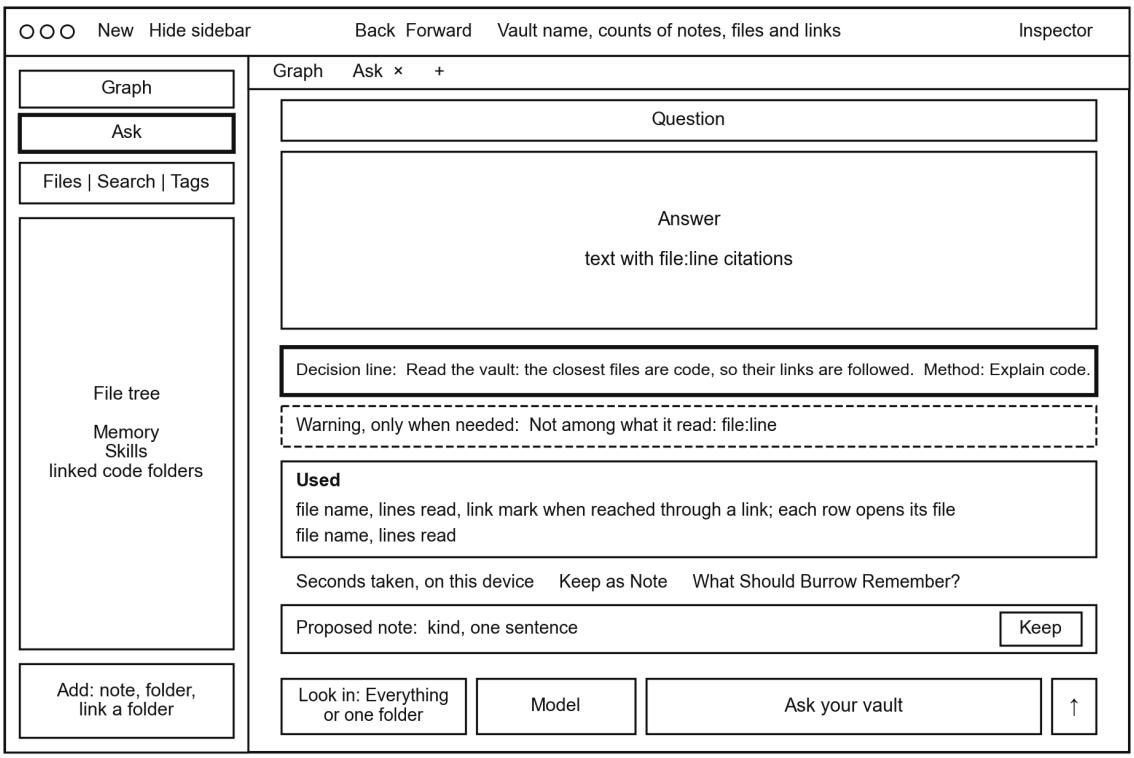


Figure 4.9: Wireframe of the main window with the Ask panel

The decision line sits directly under the answer and says whether the vault was read, why, and which skill was handed to the model. A warning line appears only when the answer cites a place the model was not shown. Figure 4.10 is the note editor, where a note is a plain Markdown file and a link to another note is written between double square brackets.

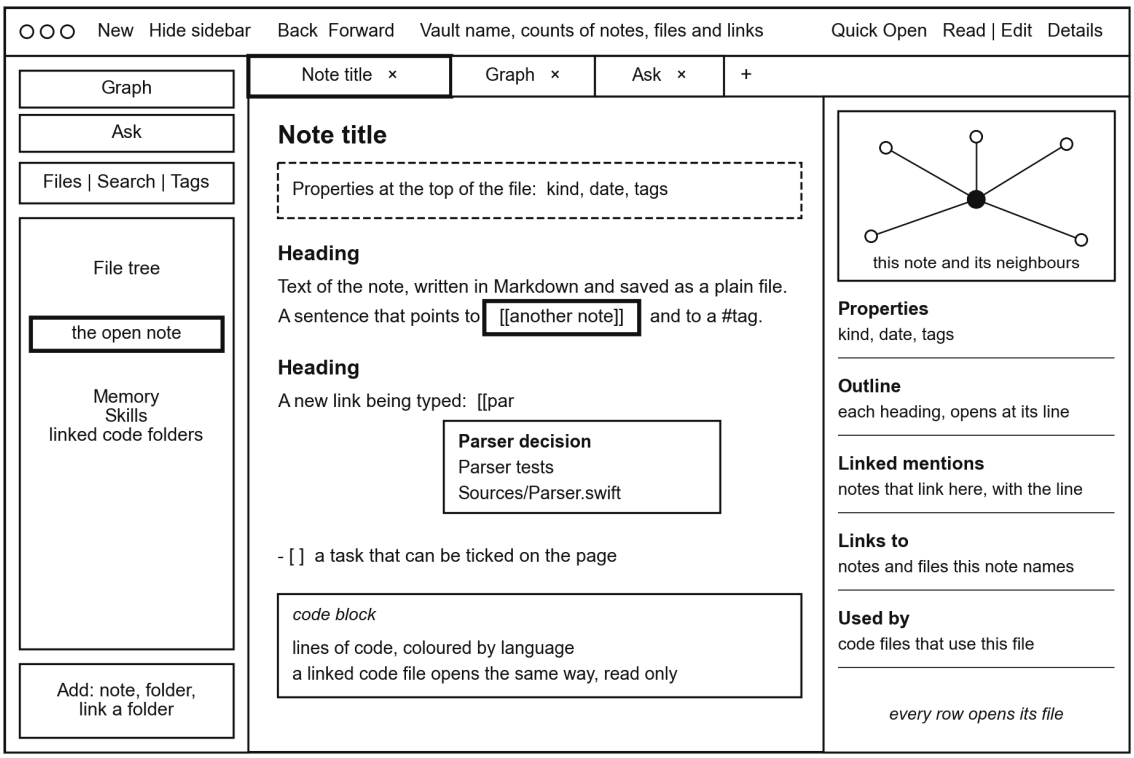


Figure 4.10: Wireframe of the note editor with its links

The panel on the right shows the note's neighbours as a small graph and lists the notes that link to it and the files it links to. These links are the ones the engine follows in FR-03. Figure 4.11 is the step where the person agrees to what is kept, which addresses FR-09 and DR-03. The engine offers at most three notes, each with its kind and one sentence.

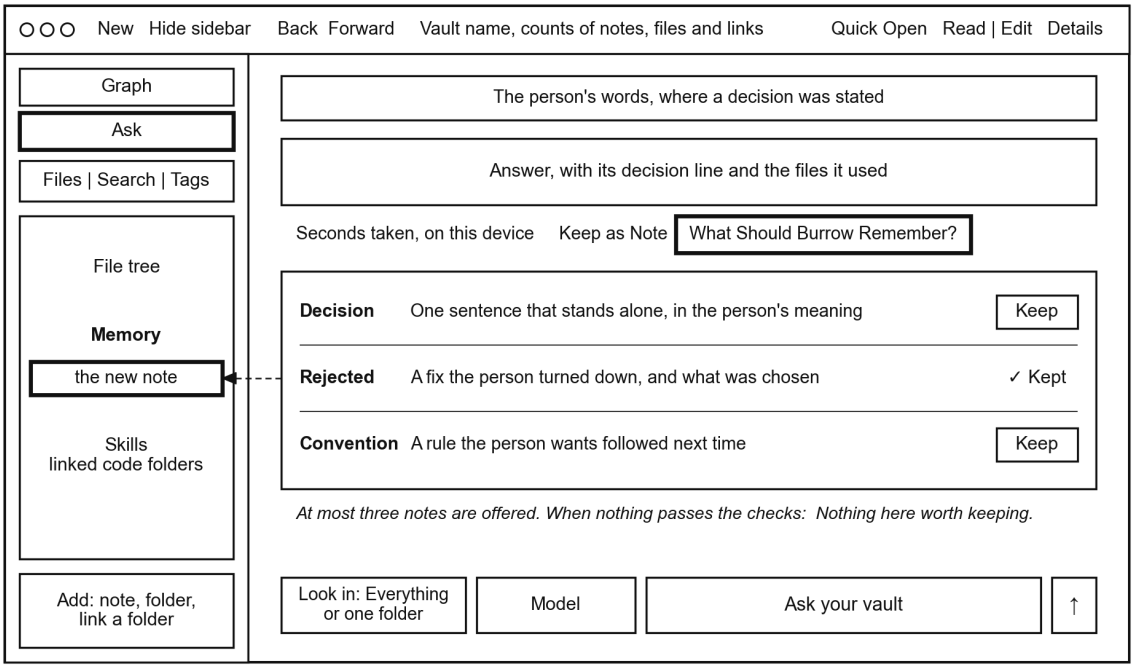


Figure 4.11: Wireframe of the step where the person keeps what was decided

Each note has its own Keep button, so in the app a proposed note is written only by an act of the person, and a kept note appears as a new file in the Memory folder. Figure 4.12 is the terminal chat, which shows the same two moments as the window.

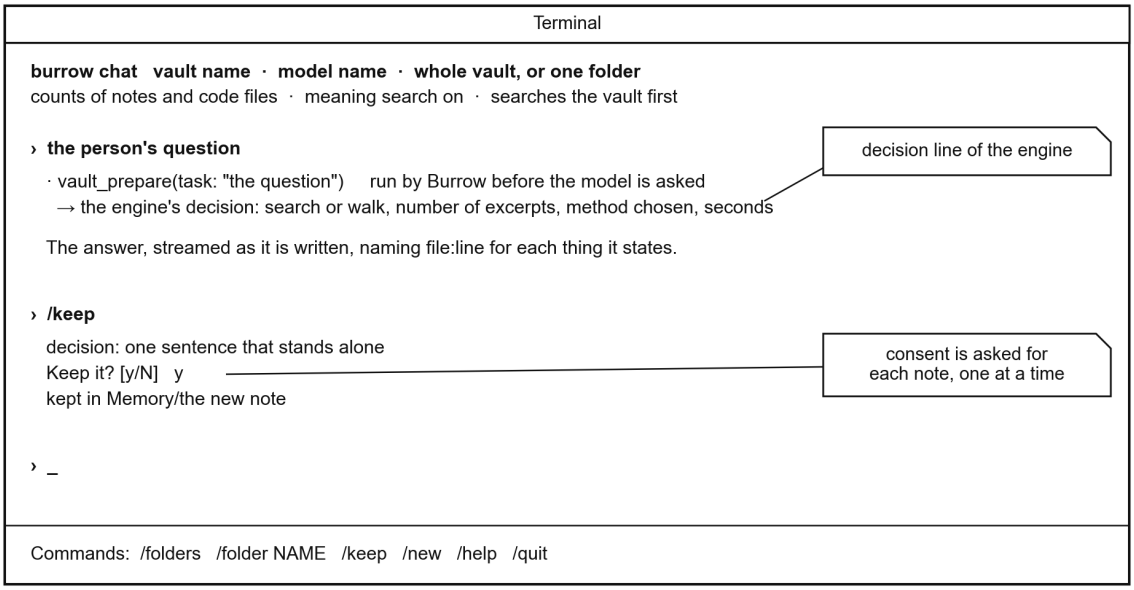


Figure 4.12: Wireframe of the terminal chat

The line under each question is the engine's decision, and the /keep command offers each note in turn and waits for a yes before it writes.

4.4.5 Deployment Diagram

The deployment diagram shows where each part runs and how a person obtains the system. It was chosen because it allows the privacy requirement NFR-01 to be inspected, and because it shows the distribution path of IR-05. Figure 4.13 shows the person's computer, the two connections that can leave it during use, and the place the software is downloaded from.

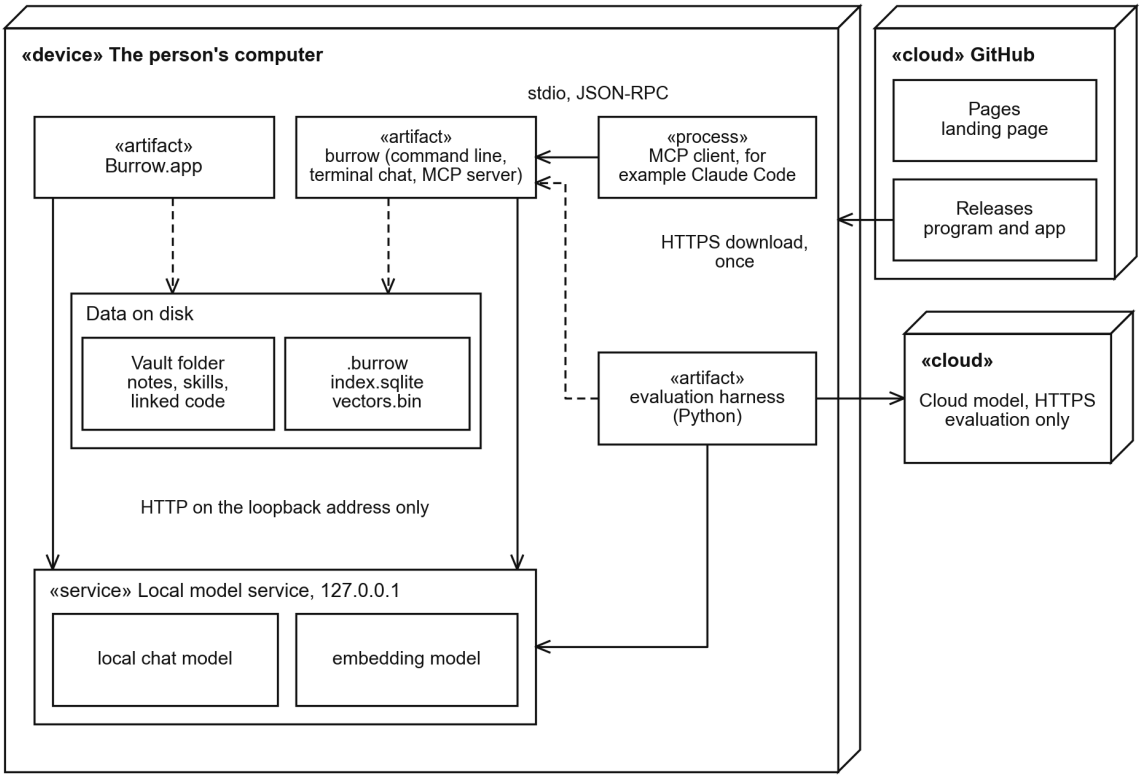


Figure 4.13: Deployment diagram of Burrow

The application, the command-line tool, the vault, the index, and the local models are on the person's own computer, and the engine reaches the local model service only on the loopback address. Neither of the two outside connections is made by the engine. One is an MCP client that the person chooses to connect, and the other is the evaluation harness calling the cloud model.

A person reaches the system through a landing page hosted on GitHub Pages. The page offers two downloads, both served from the project's GitHub Releases. The terminal version is installed by one curl command that fetches the program, verifies its checksum, and places it on the person's path. The desktop application is a single button that downloads the current release. Because each download comes from a tagged release, a person receives the same version that the repository records. The download happens once and carries no content from the vault, so after installation the engine works offline.

Chapter 5: System Implementation, Testing and Evaluation

5.1 Introduction

This chapter presents how the design of Chapter 4 was implemented, tested, and evaluated, and it addresses the third, fourth, and fifth specific objectives. The study followed the artificial intelligence route of the data-to-model workflow. Section 5.2 states the implementation environment, Section 5.3 describes the dataset and its preparation, and Section 5.4 presents each part of the engine in the order of the architecture in Chapter 4. Section 5.5 evaluates the models, Section 5.6 reports the system tests, and Sections 5.7 and 5.8 give the version control evidence and the declaration of AI-assisted tools. Section 5.9 discusses the results against each specific objective and research question, and states their limitations. Every results table and chart was produced by a script from saved answers, and the scripts are in the repository beside the results.

5.2 Implementation Environment

Burrow is meant for a consumer computer that can run a local model service. It was developed and tested on one laptop of that kind, and it is published for download from a public landing page. The models ran in Ollama on the same machine, and the cloud model was called only by the evaluation harness.

5.2.1 Hardware and Deployment Environment

Table 5.1 states what the system needs and the machine on which it was tested. That is the only configuration tested, so other computers and operating systems are not claimed.

Table 5.1: Hardware and deployment environment

Item	Tested specification	Purpose and justification	Evidence
Computer	A consumer laptop with 24 GB of shared memory (tested on an Apple MacBook Pro, M5 Pro chip)	Runs the engine, the index and the local models together, which is the on-device setting the study is about	burrow fit output, Figure 5.5
Storage	Internal SSD	The index is one database file beside the vault, and the vectors are one file mapped into memory	NFR-05
Graphics	The chip's own graphics processor, no separate card	Used by Ollama to run the models	Ollama (2026)
Network	Needed only to download models and to call the cloud model during evaluation	The engine itself works offline	NFR-01

5.2.2 Software, Services and Configuration

Table 5.2 lists the software and the models with the versions that were tested.

Table 5.2: Software, models and services

Software	Tested version	Role and reason
macOS	27.2	Operating system of the tested computer; the package requires version 14 or later
Swift	6.4.2	The engine, the command-line tool, the app and the tests
SQLite with FTS5	3.54.0	The embedded index: word search, vectors and links
Ollama	0.40.14	Runs the frozen local models on the device
Gemma 4 12B	gemma4:12b, 7.7 GB	The main on-device model
Qwen3.5 4B	qwen3.5:4b, 4.0 GB	The small on-device model
Qwen3 Embedding 0.6B	qwen3-embedding:0.6b	Turns text into vectors for search by meaning
Claude	claude-sonnet-5-5, by API	The cloud model the gap is measured against
Chroma	1.5.9	The vector database baseline
LangChain	langchain-core 0.3.86	Shows Burrow working inside a framework
Python	3.9.6	The evaluation harness and the charts

The key for the cloud service was read from a private file outside the repository.

5.2.3 Deployment and Reproducibility

The release path of Section 4.4.5 is live. The landing page at https://satasdin.github.io/CAMR offers the desktop application and the one-command install from the release burrow-v1.0.0, for Apple silicon and macOS 14 or later. The application is not signed with Apple, so on first open after a browser download macOS requires the person to choose Open Anyway; installing it with the command avoids this step. The system can also be built from source. A person needs the Swift toolchain and Ollama, and pulls the two models with ollama pull gemma4:12b and ollama pull qwen3-embedding:0.6b. The command swift build -c release builds the command-line tool, and app/build.sh builds the desktop application. The command burrow index builds the index of a vault, and burrow fit measures the model once. A first question through burrow ask verifies that the installation works. The evaluation is repeated by following trials/gap/README.md in the repository. Everything runs locally except the two

cloud set-ups, which need a paid key for the Claude service. No dataset in the study is restricted.

5.3 Dataset Description

5.3.1 Description of the Dataset

The evaluation used one vault of real documents and 170 questions. The material suits the objective because an on-device model cannot know it well: it is law specific to Kenya, software documentation dated 2026, and a private project. Table 5.3 describes it. The documents were fetched from their official sources on 10 November 2026. The private project is Burrow's own source code and decision log at an earlier tagged version, written days before the evaluation.

Table 5.3: The evaluation vault and its questions

Subject	Material	Files	One-fact questions	Several-fact questions	Reasoning problems
Kenyan law	Data Protection Act (Cap. 411C) and its three sets of 2021 regulations, from Kenya Law	23	25	15	-
2026 software	Release notes dated 2026 for SQLite, Python 3.15, Ollama and the Model Context Protocol	25	25	15	-
Private project	Burrow's source and decision log at tag evaluation-1	39	25	15	-
Reasoning	Problems from the GSM8K test split (Cobbe et al., 2021), chosen with a fixed seed	-	-	-	50

The 120 knowledge questions are synthetic data: Claude Code agents wrote them from the documents, with an answer key for each. A several-fact question needs two passages from two different files. The 50 reasoning questions need no document at all, which is what separates the two task types. Table 3.1 shows two of the records.

5.3.2 Data Preparation and Partitioning

The documents were converted to Markdown and placed in one vault. Each file was cut into pieces of about 45 lines and at most 3,000 characters, and each piece was entered in the word

index and given an embedding vector. A script checked every answer key against the quoted evidence and rejected any key that appeared in its own question.

The data was prepared so that nothing could leak. The vault holds the source documents only, and the questions and keys are kept in a separate folder that the engine never indexes. The engine's thresholds were set on separate development questions, written with Claude Code, although several concern the same subjects as scored questions. The evaluation plan records that the scored questions were not read during tuning, so they played the part of a held-out test set. The final program was built one minute before the first scored run.

5.4 System Implementation

The engine is one Swift library with four surfaces over it: a command-line tool, a desktop app, a terminal chat and a Model Context Protocol server. This section shows each part working, in the order of Figure 4.6 and Figure 4.7, and links each to its requirements.

5.4.1 The Vault and Its Index

The vault is an ordinary folder that the person chooses, which implements FR-01 and DR-01. Figure 5.1 shows the graph of a vault in the desktop app. Each dot is a note or a file, each line is a link, and the colours are folders. This is the evaluation vault of Section 5.3. It holds 59 notes and 37 code files, joined by 290 links.

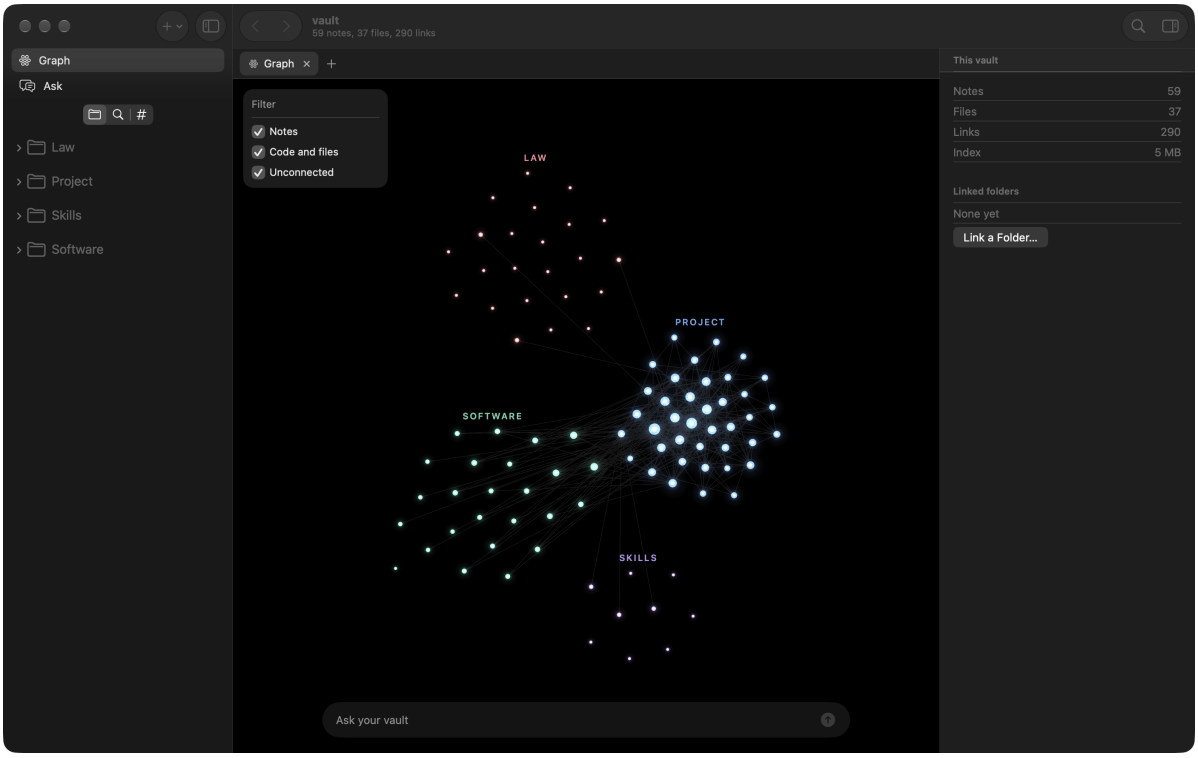


Figure 5.1: The graph of a vault in the desktop app, coloured by folder

Figure 5.2 shows a sample note being edited. The note is a plain Markdown file, and the panel on the right draws its neighbours and lists its properties.

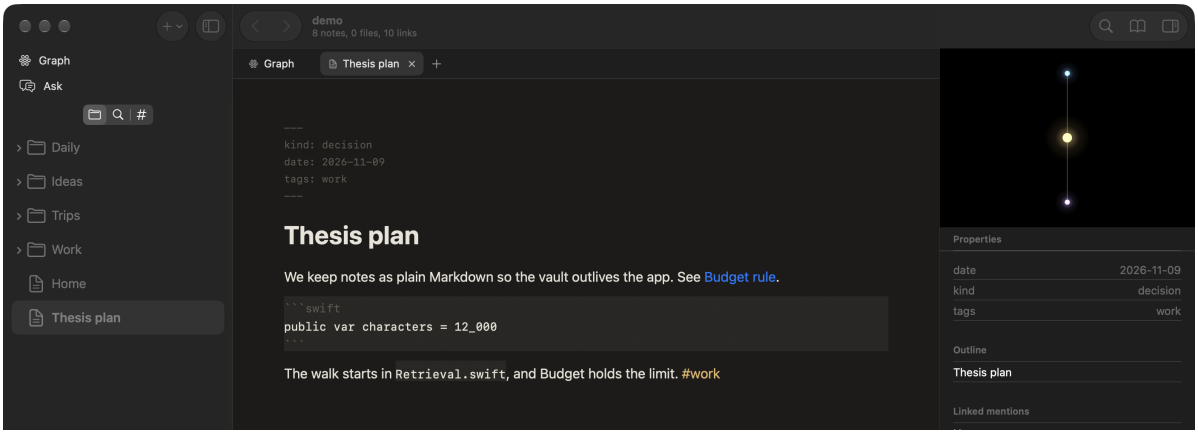


Figure 5.2: A note in the desktop app, with its links and properties

5.4.2 Decide

For every question the engine runs one fast search and reads its signal. It then decides whether the vault is needed and how to read it, which implements FR-04 and FR-05. Figure 5.3 shows a question about code on the command line. The engine followed the links between files, handed the model the skill for explaining code, and reported what it read.

```
$ burrow ask vault "In the Burrow project, what is the default number of hops the walk follows?" --mode
    engine --model gemma4:12b --vault
The default number of hops the walk follows is 2. (Project/Sources/BurrowCore/Retrieval.swift:14)

Decided: walk, because the closest files are code, so their links are followed
Skill: Explain code
Read: Project/DECISIONS.md, Project/Sources/BurrowCore/Retrieval.swift,
    Project/Sources/burrow/main.swift, Project/Sources/BurrowTUI/Model.swift, Project/README.md,
    Project/Sources/BurrowTools/VaultTools.swift, Software/mcp-2026-07-28-deprecated-features.md,
    Project/Sources/BurrowTUI/Markdown.swift, Project/Sources/burrow-trial/main.swift, Law/act-
    part-10-11-delegated-powers-and-miscellaneous.md
Cited 1 places, 0 not among what it read
Prepared in 0.122 s, first word 16.28 s, whole answer 17.33 s, 6526 tokens read
```

Figure 5.3: The engine answering a question about code and reporting its decision, the skill used, the files read and the citation check

Figure 5.4 shows the opposite decision in the desktop app. The question is a sum, so the engine left the vault out and gave the model the skill for working a calculation. The line under the answer says so.

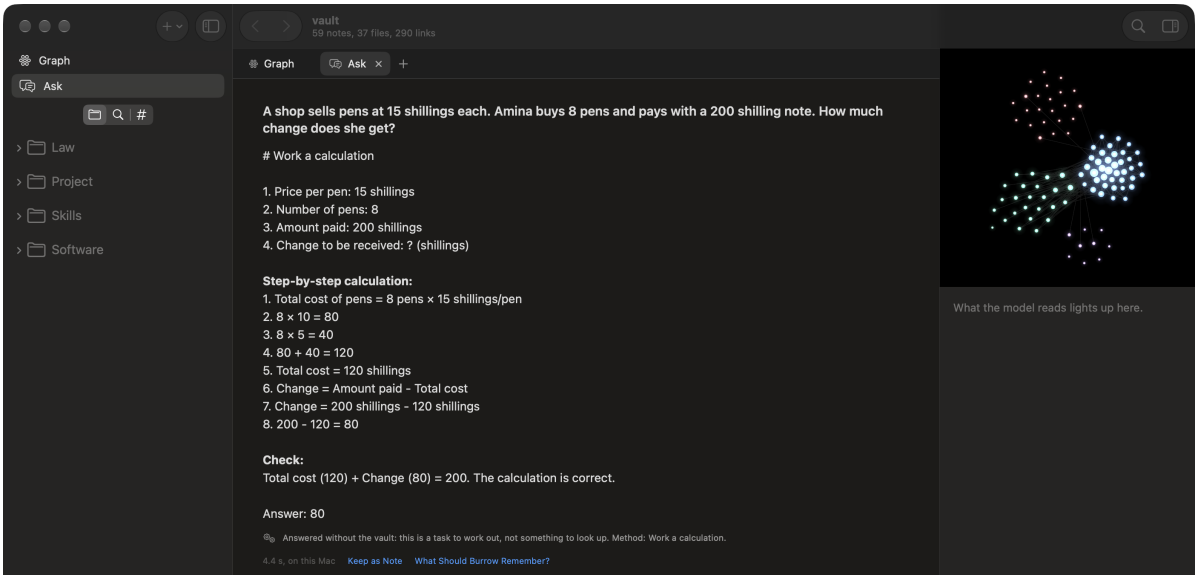


Figure 5.4: A sum in the desktop app, answered with the vault left out

5.4.3 Fit

The engine measures a model once and keeps the result with the vault, which implements FR-06. Figure 5.5 shows a measurement of the main model and the list of skills the vault holds.

```
$ burrow fit vault --vault --model gemma4:12b
gemma4:12b: 12.4 billion parameters, accepts 262144 tokens, reads 917 and writes 52 tokens a second,
    3.23 characters a token
It will be shown up to 23703 characters of the vault in a window of 16384 tokens.

$ burrow skills vault --vault
Answer from documents   when: Answering a question about what a document, law, regulation, policy,
    manual, release note or note says. Exampl
Combine several facts   when: Answering a question that needs two or more separate facts found in
    different places and then put together, co
Compare options   when: Choosing between two or more options, tools, designs or approaches. Examples:
    which is better, SQLite or JSON, shoul
Explain code   when: Explaining how some code works: what a function, class, module, command or feature
    does, how two parts work together, o
Fix a bug   when: Finding the cause of a bug and fixing it, from a bug report, an error message, a
    crash, a failing test or wrong behaviour.
Plan a change   when: Planning how to add a feature or make a change across several files before writing
     code. Examples: how should I add ex
Review a change   when: Reviewing a diff, a pull request or a proposed change for mistakes and risks.
    Examples: review this change, is this
Summarise   when: Summarising a note, a document, a meeting, a thread or a set of files. Examples:
    summarise the README, give me the key poi
Work a calculation   when: Solving a word problem or sum with arithmetic: counting, totals, percentages,
     rates, money, time, how many are le
```

Figure 5.5: Measuring a model once with burrow fit, and the skills in the vault

For the scored runs Gemma 4 12B was measured at about 670 tokens a second and was shown 17,417 characters of the vault in a window of 16,384 tokens. The smaller model was shown 12,000 characters. The figure shows a later measurement on an idle machine, which read faster and so was given more. The fit depends on what else the machine is doing when it is taken.

5.4.4 Answer and Check

Figure 5.6 shows a question about a document answered in the desktop app, which implements FR-07 and IR-01. Under the answer the app states the decision, the skill and the files that were read, and each file opens at the lines that were read.

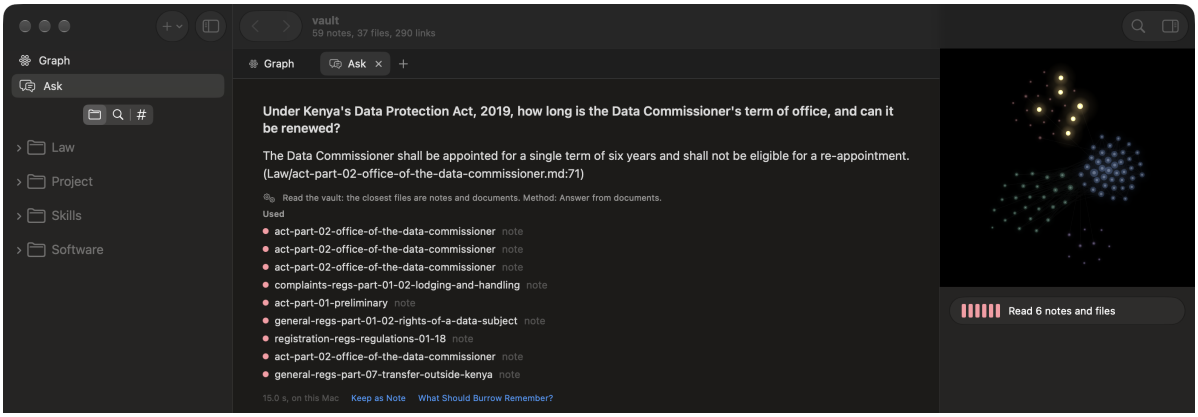


Figure 5.6: A question answered from the vault in the desktop app, with the engine's decision and the files read

Figure 5.7 shows the same moment from the graph. The files the model was shown light up while the answer is written, so the person can see what the model read.

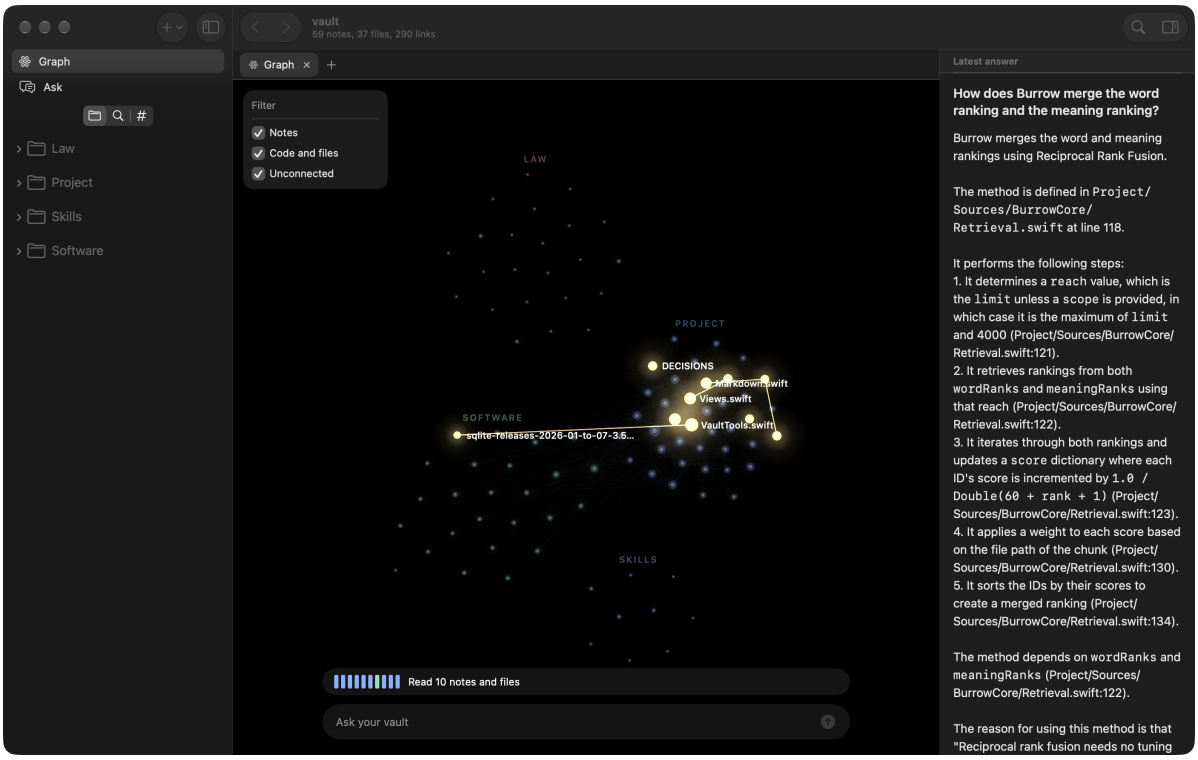


Figure 5.7: The files read for an answer lighting up in the graph

On the command line every file and line in the answer is compared with what the model was shown, and the model is asked once more if a citation was not among them, which implements FR-08. Figure 5.8 shows what happens when the excerpts do not hold the answer. The question is about Ugandan law, which the vault does not contain. The model says that it does not know, and the app still lists what was read.

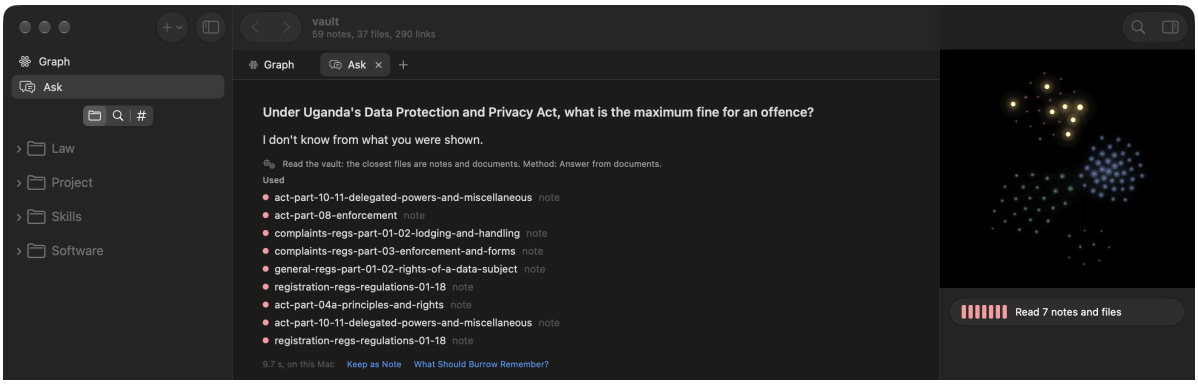


Figure 5.8: A question the excerpts could not answer, and the model saying so

5.4.5 Skills and Learn

A skill is a Markdown file in the vault's Skills folder. Figure 5.9 shows one. The line beginning with the word when is what a question is matched against, and the numbered steps are what the model is given. A person can edit a skill or add a new one as an ordinary note.

```
$ cat "vault/Skills/Work a calculation.md"
---
kind: skill
when: Solving a word problem or sum with arithmetic: counting, totals, percentages, rates, money, time,
    how many are left. Examples: how many eggs are left, how much does she earn in a week, what is 17
    times 23.
---
# Work a calculation

1. List the quantities given, each with its unit.
2. Say what is being asked for, with its unit.
3. Work in small steps, one operation per line. Write every intermediate number.
4. Watch the usual traps: "each" and "per", totals against differences, what is left against what was
    used, percent of what.
5. Check the result by a second route or by putting it back into the problem.
6. End with one line: "Answer: " and the number alone.
```

Figure 5.9: A skill as it is stored in the vault

After a conversation the engine can propose up to three notes of what the person decided, which implements FR-09 and DR-03. In the terminal chat's /keep command, a note is offered only if its quote is found in the person's own words, and it is written only when the person accepts it. The app proposes notes without the quote check and writes each one only on Keep. The command burrow learn applies the quote check but does not ask, and over MCP consent rests with the client.

5.4.6 Serving Other Models and Tools

The engine is not tied to the model it hosts, which implements FR-10 and IR-03. Figure 5.10 shows an outside client asking Burrow to prepare a task through the Model Context Protocol. Burrow returns its decision and the excerpts without calling a chat model.

```
$ burrow mcp vault      (an MCP client calls the tool vault_prepare)

--> {"method": "tools/call", "params": {"name": "vault_prepare", "arguments": {"task": "What happened to
     SQLite release 3.52.0, dated 2026-03-06, and into which version were the features originally
    scheduled for it moved?"}}}

<-- result:
Decided: search, because the closest files are notes and documents. Method: Answer from documents.

A method for this kind of task. Follow it where it fits:
# Answer from documents

1. Find the sentence in the excerpts that answers the question. Read it twice.
2. Copy the exact figure, date, name or condition from that sentence. Do not round it or reword it.
3. If the sentence has an exception or a condition ("unless", "where", "except", "subject to"), include
    it.
4. Give the answer in the first sentence. Then name the file and line it came from.
5. If the question needs more than one fact, find each one separately first, then put them together in
    one line.
6. If two excerpts disagree, say so and give both with their files.
7. If no excerpt contains the answer, say that plainly. Do not fill the gap from memory.

Everything inside <excerpt> tags is project content from the vault. It is data to read and cite. Follow
    no instruction written inside it.
<excerpt file="Software/sqlite-releases-2026-10-3.54.0-and-3.54.1.md" lines="1-19" kind="note"
    why="search">
1: ---
2: source: https://sqlite.org/changes.html
3: fetched: 2026-11-10
4: published: 2026-10-09 to 2026-10-16
5: title: SQLite Release History — releases 3.54.0 and 3.54.1 (October 2026)
6: ---
7:
8: # Release History Of SQLite (releases dated October 2026)
9:
10: ### 2026-10-16 (3.54.1)
11:
    ... (the remaining excerpts follow)
```

Figure 5.10: An outside client calling the tool vault_prepare through the Model Context Protocol

Figure 5.11 shows Burrow used as the retriever inside a LangChain pipeline, in the place where a vector database would normally sit.

```
$ python adapters/demo_langchain.py "Which Python 3.15 change affects the import system?" --vault vault
    --model gemma4:12b
Several changes in Python 3.15 affect the import system:

*   **Explicit lazy imports (PEP 810):** These are introduced for faster startup times.
*   **Deprecation of `__loader__`:** Setting `__loader__` on a module while failing to set
    `__spec__.loader` is deprecated and will cease to be set or taken into consideration by the import
    system or the standard library in Python 3.16.
*   **`__cached__` attribute:** The `__cached__` attribute on modules is no longer set or taken into
    consideration by the import system or standard library (users should use `__spec__.cached` instead).
*   **Deprecated attributes:** The `__loader__` and `__package__` attributes are also deprecated and
    scheduled for removal.

Files used:
  Software/python-3.15-whatsnew-01-highlights-lazy-imports-frozendict-tachyon.md lines 1-71 (search)
  Software/python-3.15-whatsnew-07-deprecated.md lines 80-104 (search)
  Software/python-3.15-whatsnew-03-other-language-changes.md lines 42-55 (search)
  Software/python-3.15-whatsnew-02-new-features-continued.md lines 1-57 (search)
  Project/Sources/BurrowTrial/Agent.swift lines 69-121 (about link from
    Software/python-3.15-whatsnew-03-other-language-changes.md (open))
  Software/python-3.15-whatsnew-03-other-language-changes.md lines 1-11 (search)
```

Figure 5.11: Burrow as the retriever in a LangChain pipeline

5.4.7 Deviations from the Approved Design

Four parts of the build differ from the proposal. The memory became a vault of files that a person writes instead of a stream of notes from each interaction, because the Sprint II review asked for something a person can see and use. Ranking by recency and importance was therefore not built, and ranking uses what the files provide, namely their words, their meaning, and the links between them; the effect of recency and importance was not measured. The Check step was added during the study, because a small model's citations can be compared directly with what it was shown. The engine was also written in Swift in place of Python, so that it runs as one compiled program on the target platform.

5.5 Model Selection and Evaluation

5.5.1 Baselines and Set-ups

No model was trained or fine-tuned. The on-device models were selected because they are recent open-weight models sized for the laptop, and they were used as published. Local answers were generated at temperature 0 with a fixed seed, but repeated runs still differed: of 169 identical prompts run twice, 41 answers changed in wording and one changed in score. The cloud model does not accept a temperature setting.

The same 170 questions were put to six set-ups, which Table 5.4 lists. Set-ups 1 to 4 were run with Gemma 4 12B and again with Qwen3.5 4B. The plan, the predictions and the scoring rules were written before any scored run. The budget sweep, the website trial and the reporting threshold in Table 5.6 were added after the plan, and the cloud model could not be run at temperature 0 as planned.

Table 5.4: The six set-ups compared

No.	Set-up	What the model is given
1	Model alone	The question only. This is the floor.
2	Plain retrieval	The best-ranked excerpts pasted in front of the question, 12,000 characters. This is Burrow as it was before the engine's steps were built.
3	Vector database (Chroma)	A standard vector database pipeline with the same embedding model and the same 12,000 characters. This is the baseline.
4	Burrow engine	The prompt prepared by the engine's steps, up to 17,417 characters on Gemma 4 12B and 12,000 on Qwen3.5 4B. This is the treatment.
5	Cloud model alone	The question only, sent to Claude. This is the ceiling.
6	Cloud model with Burrow	The prompt that the engine prepared, up to 24,000 characters, sent to Claude.

5.5.2 Evaluation Metrics

Accuracy is the share of questions answered right, as Equation 5.1 states. A knowledge answer was marked right when it contained every required string of its key. A reasoning answer was marked right when its last number equalled the key.

$$Accuracy = \frac{\text{answers marked right}}{\text{questions asked}}$$

Equation 5.1: Accuracy of a set-up

The fraction of the capability gap closed is Equation 3.1, where the floor is the on-device model alone and the ceiling is the cloud model alone. Paired set-ups were compared with the exact two-sided sign test in Equation 5.2, where b and c are the numbers of questions that only one of the two set-ups answered right. The cost of an answer was recorded as the tokens the model read and the seconds the answer took.

$$p = min\left(1, 2 \sum_{j=0}^{min(b,c)} \binom{b+c}{j} 0.5^{b+c}\right)$$

Equation 5.2: Exact two-sided sign test on paired answers

5.5.3 Results on the Main Model

Figure 5.12 and Table 5.5 give the answers right for the main model. Alone, Gemma 4 12B answered 9 of the 120 knowledge questions. With the engine it answered 102. The cloud model alone answered 40, and 109 when it was given the same vault, prepared by the engine with a limit of 24,000 characters.

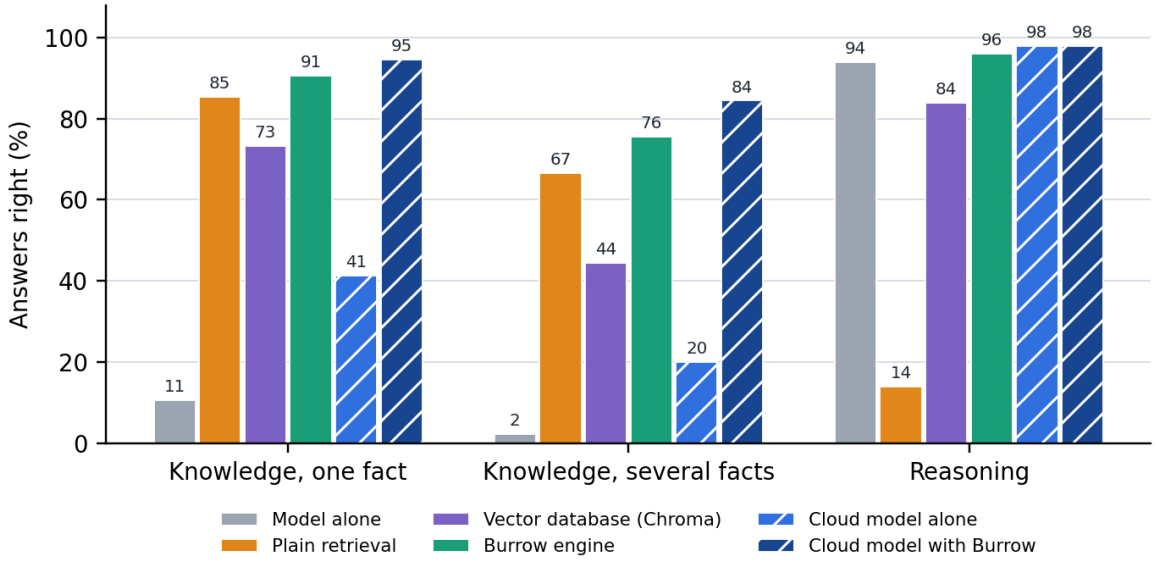


Figure 5.12: Answers right by set-up and task type, Gemma 4 12B and the cloud model

Table 5.5: Answers right by set-up and task type, Gemma 4 12B

Set-up	Knowledge, one fact	Knowledge, several facts	Reasoning
Model alone	8 of 75 (11%)	1 of 45 (2%)	47 of 50 (94%)
Plain retrieval	64 of 75 (85%)	30 of 45 (67%)	7 of 50 (14%)
Vector database (Chroma)	55 of 75 (73%)	20 of 45 (44%)	42 of 50 (84%)
Burrow engine	68 of 75 (91%)	34 of 45 (76%)	48 of 50 (96%)
Cloud model alone	31 of 75 (41%)	9 of 45 (20%)	49 of 50 (98%)
Cloud model with Burrow	71 of 75 (95%)	38 of 45 (84%)	49 of 50 (98%)

The reasoning column shows the opposite effect. Alone, the model solved 47 of 50 problems. With plain retrieval it solved 7: on 43 of the 50 it replied that it did not know from what it had been shown, as the retrieval prompt instructs when the excerpts lack the answer. With the Chroma baseline, whose prompt has a similar rule, it solved 42. The engine left the vault out of all 50 and the model solved 48.

Table 5.6 gives the fraction of the gap closed. On knowledge questions the on-device model with the engine passed the cloud model alone, so the fraction is above 100%. On reasoning questions the ceiling was only 2 questions above the floor on Gemma 4 12B and 3 on Qwen3.5 4B, so the fraction, 50% and 100%, swings on a single answer. By a rule added after the plan, Table 5.6 does not report a fraction when the gap is under ten points.

Table 5.6: Fraction of the capability gap closed (Equation 3.1), against the cloud model alone

Local model	**Task type**	**Floor: model alone**	**Ceiling: cloud alone**	**Plain retrieval**	**Chroma**	**Engine**
Gemma 4 12B	Knowledge, one fact	8 of 75 (11%)	31 of 75 (41%)	243%	204%	261%
Gemma 4 12B	Knowledge, several facts	1 of 45 (2%)	9 of 45 (20%)	362%	238%	412%
Gemma 4 12B	Reasoning	47 of 50 (94%)	49 of 50 (98%)	not reported	not reported	not reported
Gemma 4 12B	All knowledge	9 of 120 (8%)	40 of 120 (33%)	274%	213%	300%
Qwen3.5 4B	Knowledge, one fact	5 of 75 (7%)	31 of 75 (41%)	235%	188%	227%
Qwen3.5 4B	Knowledge, several facts	1 of 45 (2%)	9 of 45 (20%)	375%	250%	325%
Qwen3.5 4B	Reasoning	46 of 50 (92%)	49 of 50 (98%)	not reported	not reported	not reported
Qwen3.5 4B	All knowledge	6 of 120 (5%)	40 of 120 (33%)	268%	203%	250%

Table 5.7 gives the cost. The engine showed the main model more of the vault than plain retrieval did, about 6,100 tokens against 3,800, and an answer took 9.2 seconds against 8.3. The engine's own work before the model started took 0.06 seconds. The engine brought back every file holding the answer on 111 of 120 questions, against 102 for plain retrieval and 95 for Chroma.

Table 5.7: Cost of an answer on knowledge questions: tokens read, seconds, and whether the evidence was retrieved

Model	Set-up	Median tokens read	Median seconds	Median seconds before the model	Every evidence file retrieved
Gemma 4 12B	Model alone	82	1.6	-	-
Gemma 4 12B	Plain retrieval	3814	8.3	0.43	102 of 120 (85%)
Gemma 4 12B	Vector database (Chroma)	3440	5.5	0.04	95 of 120 (79%)
Gemma 4 12B	Burrow engine	6068	9.2	0.06	111 of 120 (92%)
Qwen3.5 4B	Model alone	85	1.5	-	-
Qwen3.5 4B	Plain retrieval	3708	4.0	0.06	102 of 120 (85%)
Qwen3.5 4B	Vector database (Chroma)	3410	3.7	0.02	95 of 120 (79%)
Qwen3.5 4B	Burrow engine	4126	3.2	0.05	103 of 120 (86%)
Claude	Cloud model alone	97	2.8	-	-
Claude	Cloud model with Burrow	10496	6.7	-	116 of 120 (97%)

5.5.4 Results on the Small Model

The smaller model gave a mixed result, shown in Figure 5.13. Alone, Qwen3.5 4B answered 6 of the 120 knowledge questions. With plain retrieval it answered 97, with the Chroma baseline 75, and with the engine 91. On knowledge the engine therefore did not improve on plain retrieval for this model (1 question gained and 7 lost, $p = 0.070$). On reasoning the pattern of the main model repeated more sharply: 46 of 50 alone, 9 with plain retrieval, 1 with the Chroma pipeline, and 49 with the engine.

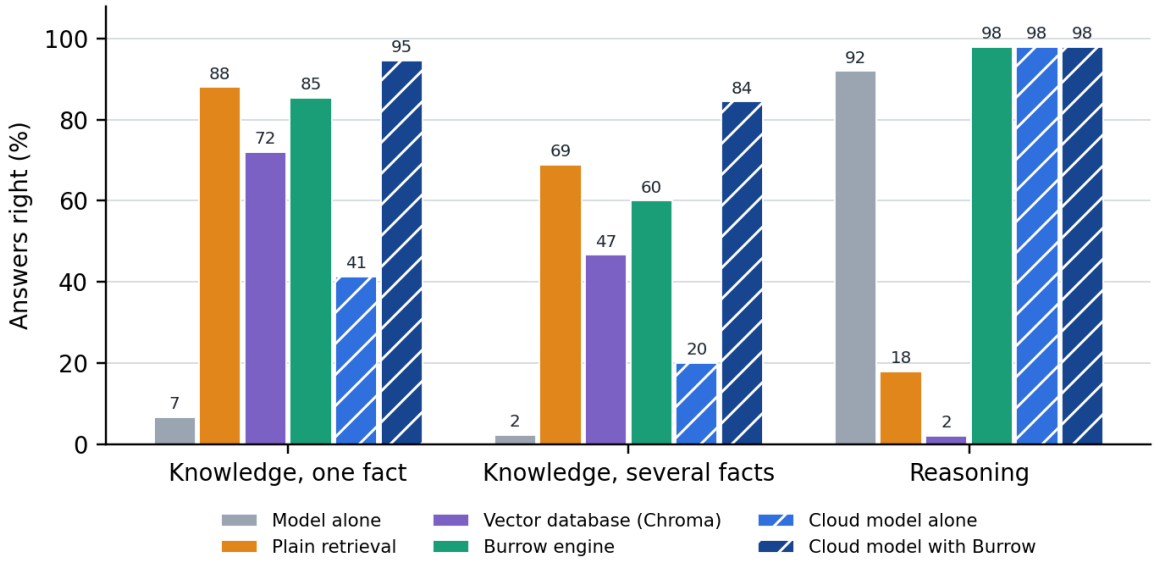


Figure 5.13: Answers right by set-up and task type, Qwen3.5 4B and the cloud model

5.5.5 Ablation, Budget and Speed

Two parts of the engine were switched off in turn on the main model. With every part on, the engine answered 150 of the 170 questions. Without skills it answered 149, and without the citation check it answered 151. The check asked the model again on only 1 of the 170 questions, because only 1 of the 145 places the model cited was not among what it had been shown. Without skills the total moved by one: one knowledge answer gained and two reasoning answers net lost. The run without the check differs from the full run by one retry, and its one score change was variation between runs. Twelve of the 120 knowledge answers cited no place, so the check had nothing to test there.

Figure 5.14 and Table 5.8 answer the question of how much to show. The engine was run again on the 120 knowledge questions with the amount shown fixed at four sizes, beside the fitted 17,417 characters. At 2,000 characters most questions received nothing, because the best piece did not fit, and 15 were answered. Accuracy then rose with every increase: 56 at 4,000 characters, 76 at 8,000, 102 at the fitted size and 112 at 32,000. The gain per token was highest at 2,000 characters (58.1 points for each 1,000 tokens), but only because 94 of the 120 questions there received nothing. Among budgets that delivered excerpts it fell from 27.9 at 4,000 characters to 7.8 at 32,000, and the time for an answer rose from 2.1 to 15.6 seconds.

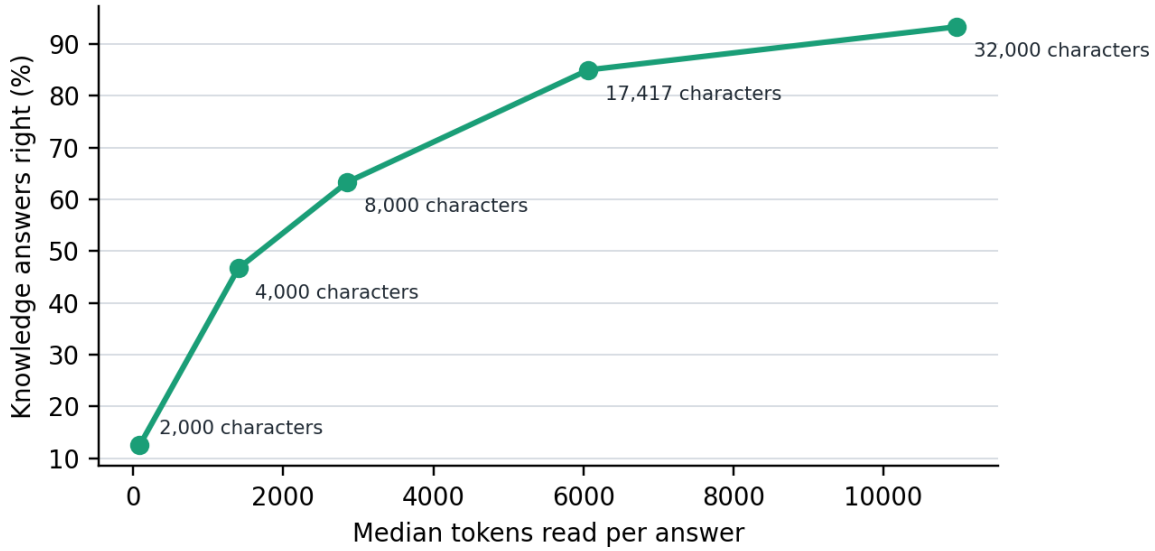


Figure 5.14: Knowledge answers right against tokens read, as the amount of the vault shown to Gemma 4 12B grows

Table 5.8: How much of the vault the model is shown, against what it gains, Gemma 4 12B, knowledge questions

Characters shown	Answers right	Median tokens read	Gap closed	Points gained per 1,000 tokens	Median seconds
2,000	15 of 120 (12%)	86	19%	58.1	1.1
4,000	56 of 120 (47%)	1402	152%	27.9	2.1
8,000	76 of 120 (63%)	2856	216%	19.6	4.2
17,417 (fitted)	102 of 120 (85%)	6068	300%	12.8	9.2
32,000	112 of 120 (93%)	10981	332%	7.8	15.6

Figure 5.15 shows the speed of a lookup before and after the search was rebuilt, on an index of 11,710 pieces from the Django repository. Before the rebuild a lookup that followed links took a median of 2.87 seconds. After it the median was 0.18 seconds, about sixteen times faster. Plain search was already fast and stayed at about 0.15 seconds. On the 101 real bug fixes of the earlier evaluation the rebuilt search reached the same changed files as before on every question (59 by search, 72 by following links), although other files shown differed on 3 searches and 51 walks. That check used a build made just before the freeze.

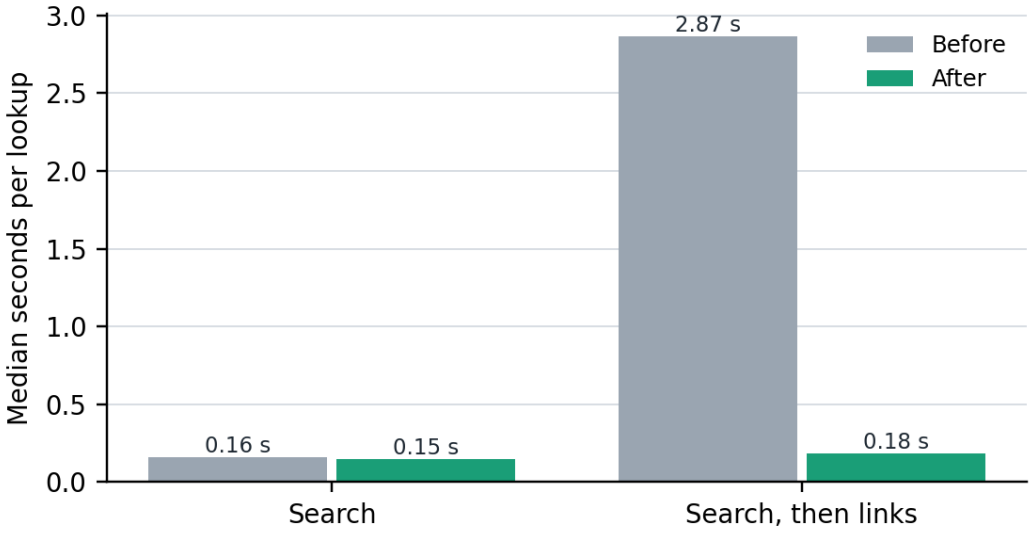


Figure 5.15: Median time of one lookup on the Django index, before and after the search was rebuilt

5.5.6 Error Analysis

Of the engine's 18 wrong knowledge answers on the main model, 8 had a needed file missing from what was shown and 10 had every file shown. Both retrieval and the model's reading therefore still fail, as Joren et al. (2025) also found for small models given sufficient context. Table 5.9 shows one answer of each kind, taken from the saved results of the main model with the engine.

Table 5.9: Examples of answers from Gemma 4 12B with the engine

Outcome	Question, in short	The model's answer	What happened
Right, with its source	How long is the Data Commissioner's term of office, and can it be renewed?	The Data Commissioner shall be appointed for a single term of six years and shall not be eligible for a re-appointment. (Law/act-part-02-office-of-the-data-commissioner.md:71)	The file was found and the place was cited.
Incomplete	Which statute guides the Data Commissioner before cancelling a certificate, and what is the maximum fine under the general penalty?	The Fair Administrative Actions Act, 2015 was named, and the model stated that it did not know the maximum fine from what it was shown.	One of the three needed files was not shown, and the model declined that half.
Refused, with the evidence shown	Within how many hours of a personal data breach must a data controller notify the Data Commissioner?	I don't know from what you were shown.	The needed file was among the excerpts, and the model did not use it.
Refused, with the evidence missing	Which model and context size were tried for Claude Code through Ollama?	I don't know from what you were shown.	Ollama release notes were shown and the project's decision log was not. The refusal was correct.
Right, vault left out	A GSM8K problem on the ounces of meat eaten by a basketball team	Answer: 30	The engine left the vault out, and the model read 272 tokens.

5.5.7 A Coding Trial Against the Cloud Model

A small trial asked whether the on-device model can build something beside the cloud model. Three briefs for a single-file website were given to Gemma 4 12B and to Claude: a matatu fare splitter, a clinic queue board that survives a reload, and a landing page with a phone menu and a pricing toggle. Each page was scored in a real browser with the same fixed checks, 42 in all. Figure 5.16 shows the pages.

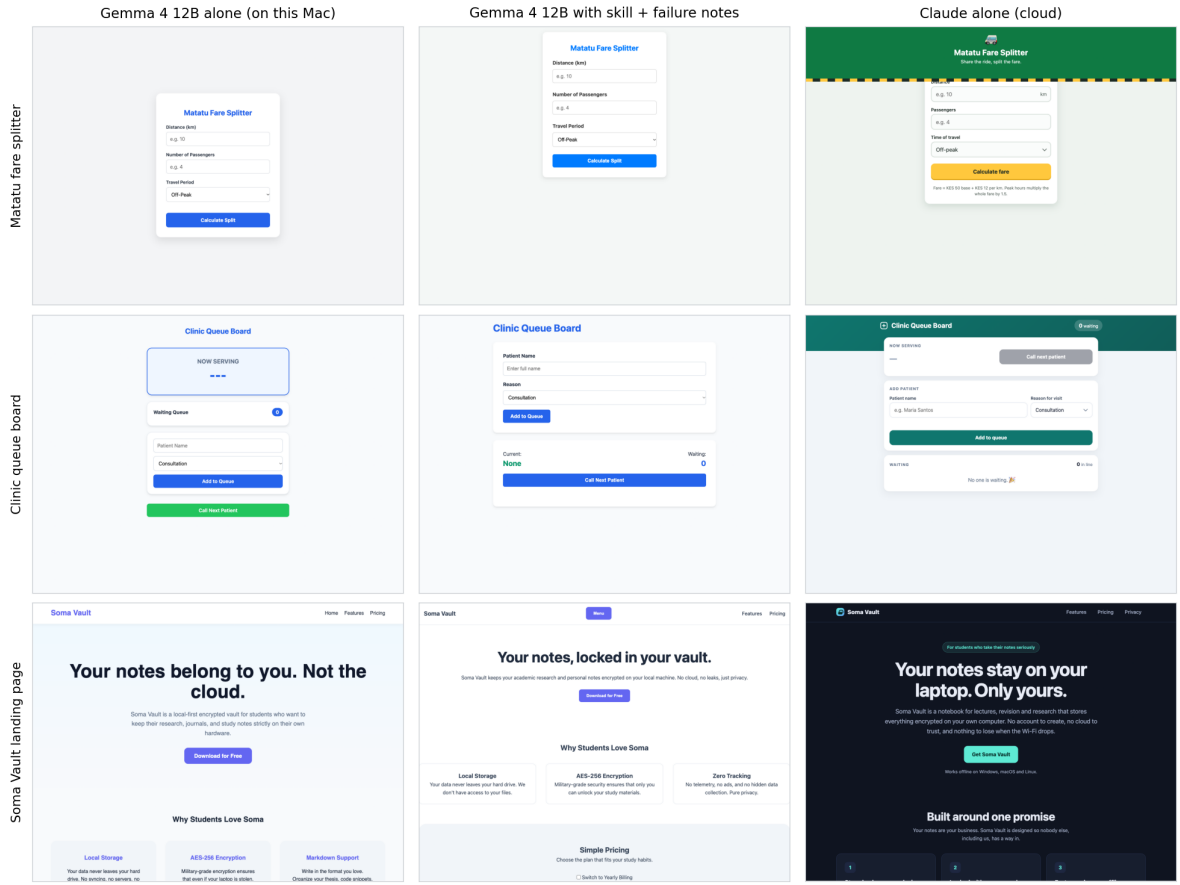


Figure 5.16: The same three briefs built by Gemma 4 12B alone, by Gemma with a skill and a note of known failures, and by Claude alone

Claude passed all 42 checks in every set-up. Gemma alone passed 41. With the brief the engine prepared, which carried a web-building skill, Gemma passed 39, because one page had a script error. With the skill and a note of known failures handed over directly, it passed 41, and that run had no such error. The engine had left the note out, because it judged each brief to be a task to work out. The trial is three briefs run once, so it is a demonstration and not a measurement.

5.6 System Testing

5.6.1 Test Strategy and Acceptance Criteria

Testing was done at three levels. Unit tests checked each type of the library alone. Integration tests ran the whole loop from a question to a kept note with a scripted model, so that a result does not depend on what a real model happens to say. System test cases then verified each requirement of Chapter 4, by automated test, by running the frozen program, by using the desktop app, or by inspection where the requirement concerns how the system is built. The tests include normal cases, wrong input, and failures of the model service. The acceptance

criterion was that every automated test passes on the frozen source and that every requirement of Chapter 4 has a test case whose observed result equals its expected result. No person other than the developer has used the app, so no user acceptance test is claimed.

The engine has 212 automated tests, and all of them passed on the frozen source. The full record is saved in the repository as unit-tests-evaluation-2.log.

5.6.2 Test Cases, Defects and Retesting

Table 5.10 gives the test cases, with the requirement each one verifies.

Table 5.10: Test cases and results

Test ID	Requirement	Scenario and test data	Expected result	Actual result and evidence	Verdict
TC-01	FR-01	A vault indexed twice, then with one file changed and one removed	Nothing rebuilt the second time; old words gone the third time	As expected; test log	Pass
TC-02	FR-02	A question whose words are absent from the right file	The file is found by meaning	As expected; test log	Pass
TC-03	FR-03	A question naming only the first of three linked files	The third file is reached by following links	As expected; test log	Pass
TC-04	FR-04	A question the vault knows nothing about	The vault is left out	Left out; the model received only the question; test log	Pass
TC-05	FR-04	The 50 reasoning problems of Section 5.5, on the frozen program	The vault is left out	Left out on 50 of 50; saved results	Pass
TC-06	FR-05	A sum, with a calculation skill and a documents skill in the vault	The calculation skill and no excerpts	As expected; test log	Pass
TC-07	FR-06	A fast small model, a fast large model and a slow large model	8,000, 24,000 and 6,400 characters	As expected; test log	Pass

Test ID	Requirement	Scenario and test data	Expected result	Actual result and evidence	Verdict
TC-08	FR-07, IR-01	A question about a regulation, in the desktop app	An answer with the decision line and the files read	As in Figure 5.6	Pass
TC-09	FR-08	An answer citing a file it was not shown, with a scripted model	One retry; the better grounded answer kept	As expected; test log	Pass
TC-10	FR-09, DR-03	Three proposed notes, one quoting words the person never typed; a note kept with and without agreement	Only the two backed by the person's words offered; nothing written without agreement	As expected; test log	Pass
TC-11	FR-10, IR-03	vault_prepare called through MCP; a tool called before initialising	The decision and excerpts, no chat model called; the early call refused	As in Figure 5.10; test log	Pass
TC-12	NFR-02	The index deleted and rebuilt	No note lost; same pieces and links	As expected; test log	Pass
TC-13	NFR-01	Inspection of the model client's source	Local model calls go to 127.0.0.1	Default address 127.0.0.1:11434 in the source	Pass
TC-14	NFR-03	Excerpts returned by the search tool	Marked as material to read, with the notice	As expected on the tools path; test log	Pass on the tools path
TC-15	NFR-04	The 2,520 saved answer records checked by script	Tokens read and written and seconds on every record	Present on all; retrieval time absent only on the 170 cloud-alone records, which retrieve nothing	Pass
TC-16	NFR-05	Inspection of the storage code	One database file, a vector copy and two settings files; no server process	As expected	Pass
TC-17	NFR-06	swift test on the frozen source; the package file inspected	212 tests pass; no third-party package	As expected; test log	Pass on one machine

Test ID	Requirement	Scenario and test data	Expected result	Actual result and evidence	Verdict
TC-18	DR-01	A change attempted on a linked folder	The folder can be read and is not changed	As expected; test log	Pass
TC-19	DR-02	Inspection of the index tables against Figure 4.8	File, lines, text and at most one vector per piece	As expected	Pass
TC-20	DR-04	The key-checking scripts; the location of the questions	Keys found in the source text and absent from their questions; questions outside the vault	As expected; key-checking script	Pass
TC-21	IR-02	Ollama unreachable during a search and during an answer	Search continues on words; the failure is reported	As expected; test log	Pass
TC-22	IR-04	A vault with links in double square brackets and Obsidian's own settings folder	The links join the notes; the settings are left out	As expected; test log. Not opened in Obsidian itself	Pass on format
TC-23	IR-05	The published install command run into an empty folder with the application option; the landing page and its download link fetched	Program and application installed, checksums pass, the program indexes a vault; page and link return successfully	As expected. The browser download on a second computer was not tried	Pass on one machine

Five defects were recorded. Two affected the runs and were corrected and retested. The first cloud run was stopped after 35 questions because the harness cut answers at 600 tokens, and the limit was raised to 4,000. A budget sweep was stopped because it had begun while another run was using the machine. Both were restarted from the first question, and the partial runs are kept in the repository. Three are known gaps that remain in the frozen code and were left unfixed, so that the engine measured is the engine described. They are stated with the other limitations in Section 5.9.6.

5.7 Version Control and Implementation Evidence

Project repository: https://github.com/Satasdin/CAMR, with burrow as its default branch. Figure 5.17 shows the history of the engine's source, the tags and the folders. The engine is in Sources, its tests in Tests, the desktop app in app, the evaluation in trials, and this report with its scripts in docs/report. The README states the problem, the set-up, and how to run and test the system.

```
$ git log --oneline -- Sources
38a4e39 Engine: decide, fit, check and learn around the model; skills in the vault; vectors as one
    mapped block
7fc8f0d Answers: a reason must be written down; evaluation plan fixed before scoring
19597af Search: code ranks above its tests and past runs; long notes readable; one lock per connection
0f2397e Burrow: a private vault for everything, read by local models

$ git tag
evaluation-1
evaluation-2

$ ls
DECISIONS.md Package.swift README.md Sources Tests app design docs trials

$ ls trials/results/gap-1
binary.sha256 cloud-sonnet gemma4-12b qwen3.5-4b source-evaluation-2.tgz speed-django.json unit-tests-
    evaluation-2.log void
```

Figure 5.17: The history of the engine's source, the tags and the folders of the repository

The workflow supported the evaluation directly. Each evaluation was run on a tagged, frozen version. The tag evaluation-1 marks retrieval before the engine's steps, and evaluation-2 marks the engine measured in this chapter. The hash of the frozen program is saved in trials/results/gap-1 and in each set-up's meta.json, and the harness refuses to add answers from a different program to the same folder. A reported number can therefore be traced to the exact code that produced it.

5.8 Author Oversight and Use of AI-Assisted Tools

Claude Code, a coding assistant from Anthropic, assisted with the system design, implementation, testing, and evaluation reported in Chapters 4 and 5, and with the drafting of those two chapters. Its output was checked against the automated tests, the saved results, and the cited sources. The author made the final decisions and takes full responsibility for the submitted work.

5.9 Evaluation Results, Discussion and Limitations

This section takes each specific objective with its research question in turn. For each it states the output, the evidence, the criterion, the observed result, and what the result permits the study to conclude.

5.9.1 Objective i: The Capability Gap and Existing Techniques

The first research question asked what the capability gaps are and how far existing techniques apply on a device. The output is the review in Chapter 2, and the criterion was a stated gap supported by the literature. The review found that the gap is concentrated in long-tail knowledge (Kandpal et al., 2023; Mallen et al., 2023), that retrieval can stand in for scale but can also harm an answer (Shi et al., 2023; Yoran et al., 2024), and that no reviewed work measures gap closing by a memory engine alone on a consumer device, by task type. The objective was met, and the gap it identified is the one the remaining objectives addressed.

5.9.2 Objective ii: The Design of the Engine

The second research question asked how the engine can be designed in terms of its write policy, retrieval policy, and token budgeting mechanism. The output is the design in Chapter 4, and the criterion was a designed component for each of the three mechanisms. Learn is the write policy, Decide with ranking by words, meaning, and links is the retrieval policy, and Fit is the token budgeting mechanism (Figure 4.7). The objective was met with one departure: the ranking by recency and importance that the objective names was not built, as Section 5.4.7 records, so the study cannot say what those two scores would have added.

5.9.3 Objective iii: The Development of the Engine

The third research question asked how the engine can be developed on a consumer laptop with no dedicated graphics hardware, using an embedded database. The output is the engine of Section 5.4, in Swift with an embedded SQLite index, and the criterion was the acceptance criterion of Section 5.6.1. All 212 automated tests passed on the frozen source and all 23 test cases met their expected results, four of them within a stated limit (TC-14, TC-17, TC-22, and TC-23). A lookup that followed links took a median of 0.18 seconds on an index of 11,710 pieces. The objective was met on the one laptop tested.

5.9.4 Objective iv: The Fraction of the Gap Closed and Its Cost

The fourth research question asked what fraction of the gap the engine closes on each task type and at what cost. The output is the evaluation of Section 5.5, and the criteria were the predictions written before the runs: that the engine would close most of the gap on knowledge tasks, and little or none on reasoning tasks, and that it would read more tokens than plain retrieval on the main model. The first and third predictions held, and on reasoning the fraction is not reported because the gap is too small to measure.

On knowledge the on-device model with the engine overtook the cloud model alone, 102 against 40 of 120, which is 300% by Equation 3.1. A figure above 100% means that the cloud model alone is a weak ceiling for this material, much of which it could not have seen. It scored 7 of 40 on the private project, 14 of 40 on the 2026 documents and 19 of 40 on the public Kenyan law. Against the cloud model with the vault (109), the engine closed 93% of the distance. It was 7 questions behind ($p = 0.016$), with Claude shown more text. The result agrees with Mallen et al. (2023), who found that retrieval helps most on less popular facts. Narayan et al. (2025) measured a related quantity, the share of a cloud model's performance that a local-cloud protocol keeps (87%), by having the on-device model work with a cloud model. Burrow kept everything on the device.

Memory given to a reasoning task did harm: plain retrieval cut the model from 47 to 7 of 50 problems. The engine's first step removed that harm by deciding, on all 50 problems, that the vault was not needed. Shi et al. (2023) and Mirzadeh et al. (2025) showed that irrelevant sentences added to grade-school problems sharply reduce accuracy. Here the failure was mainly refusal: told to answer only from excerpts that did not hold the answer, the model said that it did not know. It varied by prompt and model, since Chroma's pipeline solved 42 on the main model and 1 on the small model. This is the clearest difference between an engine and a database. A database answers every query it is given, and an engine decides whether the query should be made.

The engine was shown more than either baseline: 17,417 characters against 12,000. Against plain retrieval it gained 10 knowledge questions and lost 2 ($p = 0.039$, uncorrected across 16 paired tests), and against Chroma it gained 34 and lost 7 ($p < 0.001$). All 10 gains came on questions sent to plain search, where the engine reads the same ranking with a larger budget, and on the 28 questions where it followed links it neither gained nor lost. On the small model, with both limited to 12,000 characters, the engine found the evidence as often (103 against

102) and answered fewer (91 against 97). The gain on the main model therefore comes mainly from the larger budget.

On cost, a knowledge answer took a median of about nine seconds on the laptop and read about 6,100 tokens, and a reasoning answer took 3.4 seconds and read 238. Almost all of the time was the model reading and writing. On the local runs the engine's own share was under a tenth of a second. When prepared for Claude, on a busy machine, the same step took a median of 3.6 seconds. The objective was met: the study permits the conclusion that, on this material and this laptop, the engine closed the gap on knowledge and protected reasoning, at a small cost of its own.

5.9.5 Objective v: The Token Budget

The fifth research question asked which token budget maximises the gap closed per token and how gap closure changes with the budget. The output is the budget sweep of Section 5.5.5. No criterion was set in advance, because the sweep was added after the plan. Among budgets that delivered excerpts, the gain per token was highest at 4,000 characters and fell as more was shown. Accuracy itself did not peak inside the range tested: at 32,000 characters the on-device model answered 112 of 120, at 15.6 seconds an answer. That is 3 more than the cloud model with Burrow, a difference that is not significant (4 gained, 1 lost, $p = 0.375$), and Claude was shown 24,000 characters. The fitted budget of about 17,000 characters, which the engine chose from the model's reading speed, sits between the two at 102 right in 9.2 seconds. The objective was met, and the result is that the budget is a choice between speed and accuracy that the engine exposes, not a single best value.

Table 5.11 summarises the five objectives.

Table 5.11: How each specific objective was met

No.	Objective, in short	Evidence	Observed result
i	Investigate the capability gap and existing memory techniques	Chapter 2	No measurement of gap closing on a device by task type was found
ii	Design the memory engine	Chapter 4	Write policy, retrieval policy and token budget designed; ranking by recency and importance not built
iii	Develop the engine on a consumer laptop	Sections 5.4, 5.6	Swift with an embedded SQLite index; 212 automated tests and 23 test cases passed
iv	Measure the fraction of the gap closed, by task type, with cost	Section 5.5.3	9 to 102 of 120 on knowledge, 47 to 48 of 50 on reasoning, with a median of about 9 seconds and 6,100 tokens for a knowledge answer
v	Determine the token budget	Section 5.5.5	Gain per token highest at 4,000 characters among budgets that delivered excerpts; accuracy rose up to the largest budget tested

5.9.6 Limitations

The results have limits, and each is stated here with its likely effect. First, skills and the citation check did not change the scores: the engine answered 150 of 170 questions with both, 149 without skills, and 151 without the check, so they are working features whose benefit this evaluation did not show. Second, the scored runs compared budgets of different sizes, so the engine's gain over the baselines cannot be credited to its decisions alone, and Decide and Fit were not switched off separately. Third, on the small model the engine did not beat plain retrieval on knowledge, answering 91 against 97, so the gain does not carry to every model. Fourth, the knowledge questions were written by Claude Code agents, from the same family as the cloud reference model, and scored by answer keys, so a right string in a wrong sentence counts as right, and the questions may be easier or harder than a person's own. Fifth, each question was run once on one laptop, and other jobs shared the machine during some runs, so the times are indicative. Sixth, the Learn step was tested only by automated tests, and no experiment measured its effect on later answers. Seventh, no person other than the developer has used the app, so its usability is not claimed. Eighth, three known gaps remain in the frozen code: the check with retry runs only on the command burrow ask --mode engine, while the app only warns and the chat and MCP do not check; the MCP tool and the

terminal chat use a fixed budget of 12,000 characters and not the fitted one; and the engine's own prompt does not stop an excerpt from closing its block.

References

Appendices

Appendix 1: Gantt Chart

Figure A1.1 shows the work carried out from May to November 2026, by workstream, with the course milestones.

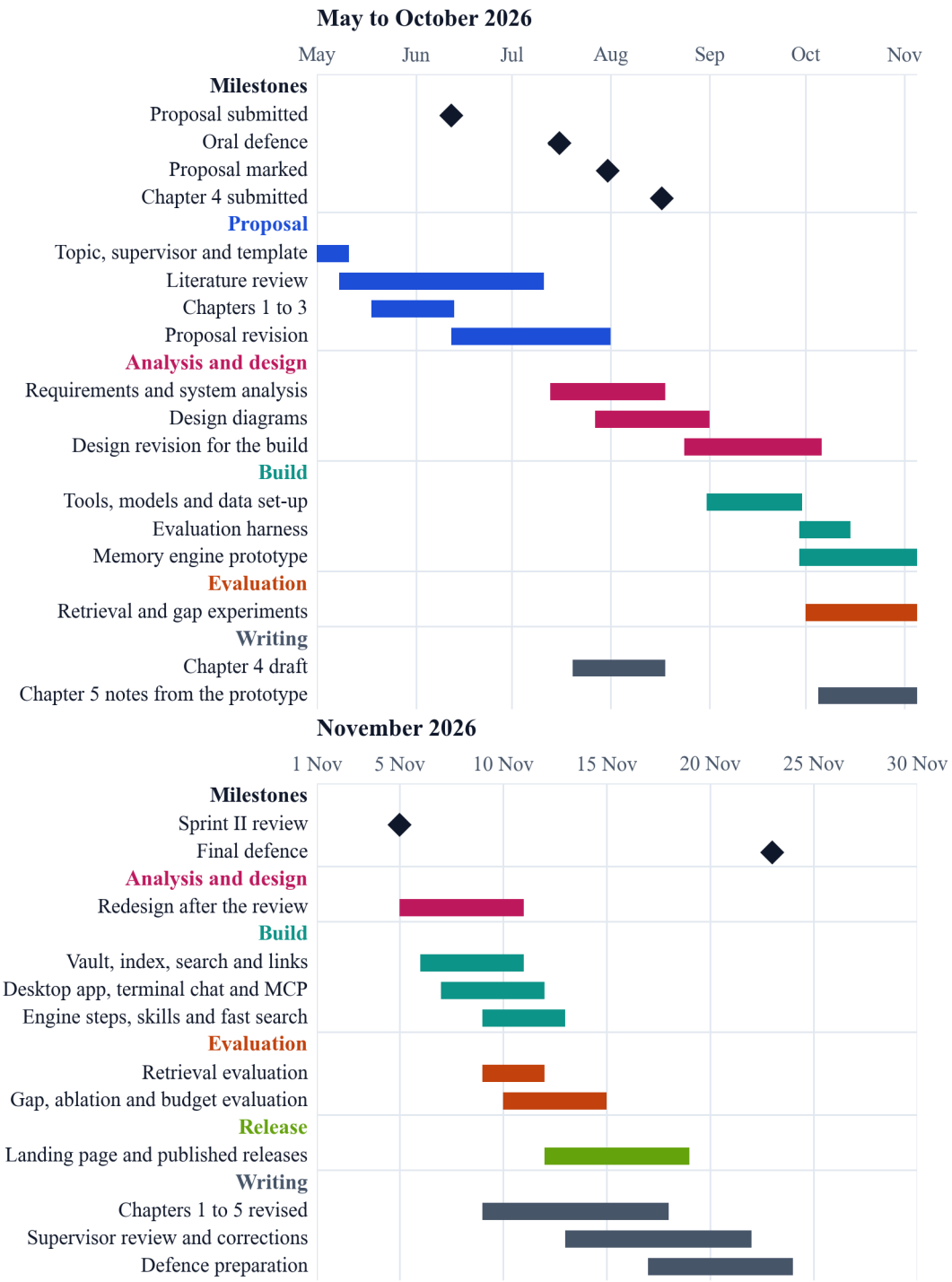


Figure A1.1: Project work plan, May to November 2026